

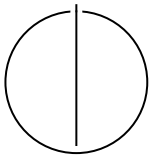
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — CHAIR OF DISTRIBUTED SYSTEMS AND OPERATING SYSTEMS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

GPUDirect-enabled Confidential Virtual Machines via Trusted SmartNICs

Simon Weckler





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — CHAIR OF DISTRIBUTED SYSTEMS AND OPERATING SYSTEMS

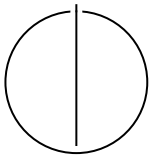
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

GPUDirect-enabled Confidential Virtual Machines via Trusted SmartNICs

GPUDirect-fähige Confidential Virtual Machines mittels attestierter SmartNICs

Author:	Simon Weckler
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisor:	Teofil Bodea
Submission Date:	September 18, 2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.



Munich, 18.09.2026 Simon Weckler

Acknowledgments

I want to thank Prof. Dr. Pramod Bhatotia for supervising this thesis and allowing me to explore this topic.

I would also like to extend my deepest thanks to my advisor Teofil Bodea for supporting me through this thesis. His guidance and expertise were invaluable to me and the final product.

Abstract

Modern machine learning workloads increasingly span GPU clusters, making inter-GPU communication critical to performance. GPUDirect RDMA provides a direct path without staging payload through host memory, but Hopper Confidential Computing blocks NIC access to protected GPU memory. The secure fallback, therefore, stages payloads in CPU memory and undergoes repeated software-based cryptography. We present **IronLink**, an architecture for secure GPU-to-GPU RDMA between confidential virtual machines via two device-side data paths. **In-GPU Bounce** seals records with GPU Copy Engines before GPUDirect transfer. **Trusted NIC** instead admits an attested NIC and secures the PCIe and network links with IDE and ESP. IronLink defines the common attested control plane and hardware interfaces required by both designs. Because current hardware lacks this support, we evaluate both as projections using microbenchmarks, cost models, and distributed-ML simulations. Across selected workloads, In-GPU Bounce and Trusted NIC achieve projected median speedups of 4.3× and 5.2× over the CPU bounce, with median overheads of 23% and 2% over GPUDirect, respectively. Source code and artifacts are available at github.com/TUM-DSE/gpu-network-io.

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**

2 Background **3**
2.1 Confidential Virtual Machines . . . 3
2.2 Confidential GPUs . . . 4
2.2.1 Command Submission . . . 4
2.2.2 Copying Data . . . 5
2.2.3 Copy-Engine Key and IV Multiplexing . . . 5
2.3 GPUDirect RDMA . . . 6
2.3.1 PCIe Peer Transactions . . . 7
2.4 Trusted I/O . . . 7
2.5 IPsec and ESP Offload . . . 10
2.6 Modeling Communication Cost . . . 10

3 Motivation **12**
3.1 GPUDirect RDMA is Incompatible with Confidential GPUs . . . 12
3.2 Fallback: Bounce Buffer Through CPU Memory . . . 13

4 Overview **16**
4.1 Design Goals . . . 16
4.2 Threat Model . . . 17
4.3 The IronLink Architecture . . . 17
4.3.1 Trust and Session Establishment . . . 18
4.3.2 IronLink Channel Creation . . . 18
4.3.3 Record Transfer . . . 19
4.3.4 Invalidation and Re-establishment . . . 19
4.4 Design Alternatives . . . 19

5 Design 1: In-GPU Bounce **21**
5.1 Session Manager . . . 22
5.2 Orchestrator . . . 23
5.2.1 IronLink Channel . . . 23

5.2.2 Channel Creation 23
5.2.3 Communicator 24
5.2.4 Channel Progress Engine 24
5.3 GPU and NIC Interfaces 26
5.4 Datapath and Cost Model 26
5.4.1 Bounding Copy Engine bandwidth 27

6 Design 2: Trusted NIC **28**
6.1 Session Manager 29
6.1.1 Device Trust Manager 29
6.1.2 Data Session Broker 33
6.2 Orchestrator 33
6.3 GPU and NIC Interfaces 34
6.4 Datapath and Cost Model 34
6.4.1 Estimating fixed latency 35
6.4.2 Estimating asymptotic bandwidth 35

7 Evaluation **37**
7.1 Methodology 38
7.1.1 Measurements 38
7.1.2 Parameter Sources and Assumptions 39
7.2 Bandwidth 40
7.3 Latency 42
7.4 End-to-End Flow Completion Time 43
7.4.1 Model Instantiation and Validation (Q1) 44
7.4.2 Datapath Comparison (Q2) 46
7.5 Collective Communication (Q3) 47
7.5.1 Simulation Setup 47
7.5.2 Collective Completion Time 48
7.6 Distributed ML Workloads (Q4) 51
7.6.1 Simulation Setup 51
7.6.2 Results 53
7.6.3 Interpretation 54

8 Discussion **55**
8.1 Limitations 55
8.2 Future Directions 56

9 Related Work **58**

10 Conclusion **60**

A NVIDIA Driver Source Excerpts **61**

Abbreviations **63**

List of Figures **66**

List of Tables **67**

Bibliography **68**

1 Introduction

Modern machine learning systems increasingly distribute training and inference across GPU clusters. Their performance depends not only on computation, but also on moving parameters and gradients between accelerators [52, 61]. GPUDirect Remote Direct Memory Access (RDMA) reduces this cost by allowing a Network Interface Card (NIC) to access GPU memory directly, avoiding host-memory staging and CPU payload processing [11].

Confidential Computing protects data while it is processed on otherwise untrusted cloud infrastructure. Confidential Virtual Machines (CVMs) isolate guest memory from the host, while NVIDIA Hopper extends this model to GPUs through protected GPU memory, device attestation, and authenticated command submission [4, 21, 28]. These protections, however, conflict with GPUDirect RDMA. Hopper's firewall prevents the NIC from accessing the Compute-Protected Region (CPR), and the platform exposes no trust-establishment mechanism required for direct peer access [21, 28, 58]. The secure fallback, therefore, copies each payload through CPU memory at both endpoints, performs six cryptographic passes over the payload (four in CPU software and two in the GPU Copy Engines), adds host-memory traffic, and pays confidential GPU command submission costs for every transfer.

Prior work addresses individual parts of this problem. Confidential GPU systems protect execution or optimize CVM-to-GPU transfers [71, 74, 75, 81], while SCALE co-designs encrypted communication and collective algorithms [60]. sRDMA offloads RDMA authentication and optional encryption to a trusted NIC [76]. Trusted-I/O mechanisms provide device attestation and protected local I/O paths [3, 13, 62, 63]. To the best of our knowledge, however, existing approaches do not compose confidential GPU endpoints, GPU-resident inter-node RDMA, protected local and network paths, and device trust into a complete end-to-end design. This missing composition forms the research gap addressed.

To close this gap, we ask how confidential GPUs in different CVMs can communicate through RDMA while preserving confidentiality and integrity. The desired path should keep plaintext within attested components, remove the CPU from payload processing, avoid staging through host memory, and minimize fixed per-transfer overhead.

We present **IronLink**, an architecture for secure GPU-to-GPU RDMA that separates a CVM-managed control path from a device-side payload path. **Design 1, In-GPU Bounce**, keeps the NIC outside the Trusted Computing Base (TCB): the sending Copy Engine seals a record into unprotected GPU memory, standard GPUDirect RDMA transfers the ciphertext, and the receiving Copy Engine unseals it into the CPR. **Design 2, Trusted NIC**, admits an attested NIC into the TCB, grants it restricted access to an Exposure Buffer in the CPR, protects GPU-to-NIC traffic with PCIe Integrity and Data Encryption (IDE), and protects the network with NIC offloaded Internet Protocol Security (IPsec) Encapsulating Security Payload (ESP).

Because current GPU firmware and NIC hardware do not provide all the capabilities required by IronLink, the complete data paths remain projections. Instead, we implement the measurable building blocks through RDMA perftest extensions, a driver-level benchmark for sealed Copy Engine transfers and command latency, latency-bandwidth models, and ASTRA-sim experiments using STAGE-generated distributed ML traces. On our NVIDIA H100 and ConnectX-7 testbed, a sealed Copy Engine transfer reaches 38.78 GB/s, exceeding the measured 23.15 GB/s GPUDirect rate. For a 32 B confidential copy, command submission contributes 17.31 µs or 89% of its fixed latency. Across the selected workload traces, Design 1 and Design 2 achieve a projected median speedup of 4.3× and 5.2× over the CPU bounce, with median overheads of 23% and 2% over GPUDirect.

IronLink shows that removing CPU software cryptography allows both projected designs to reach the measured GPUDirect bandwidth on our testbed, where communication is network-bound. Confidential command submission, however, remains the key obstacle to low-latency communication.

Contributions. This thesis contributes: (i) **Problem characterization**. We explain why GPUDirect RDMA conflicts with Hopper confidential computing and characterize the fallbacks' overheads; (ii) **IronLink architecture**. We design a secure inter-node GPU communication architecture with two realizations: In-GPU Bounce and Trusted NIC, which trade TCB size for performance; and (iii) **Evaluation**. We build measurement, modeling, and simulation infrastructure to project flow, collective, and distributed workload performance.

2 Background

2.1 Confidential Virtual Machines

We use the term CVM to denote a VM Trusted Execution Environment (TEE) instance. Intel Trust Domain Extensions (TDX), AMD Secure Encrypted Virtualization (SEV)-Secure Nested Paging (SNP), Arm Confidential Compute Architecture (CCA), and IBM Protected Execution Facility (PEF) are examples of CVM technologies: they place a complete VM, including its guest OS, inside an isolated execution context [72]. Unlike conventional virtualization, a CVM does not rely on a trusted hypervisor to enforce isolation. Hardware and security firmware protect the guest's execution state and memory while the untrusted host continues to schedule the VM and manage resources [72].

CVM Memory. CVMs distinguish between private and shared system memory. Private memory lies within the CVMs' protected memory domain, while shared memory is deliberately exposed for communication with the host or devices. Assigning a peripheral to a CVM does not make the device trusted, so an ordinary device's DMA cannot access private memory. Without Trusted I/O, data must instead be staged through shared system memory [4, 9, 72]. Once exposed in shared memory, data is outside the CVM's Trust Boundary and must be protected by the VM when confidentiality or integrity is required [4, 9]. Trusted I/O removes this restriction by establishing a protected path and access-control rules that allow an attested device to access memory, as described in Section 2.4 [13].

Remote Attestation. Remote attestation lets an external party verify a CVM before establishing trust and a secure channel. The hardware-rooted TCB reports an integrity-protected measurement of the CVM's initial state and TCB, which the verifier checks and compares against expected values [72]. The report can also bind a guest-supplied public key to the measured CVM [72]. The peer channel in Section 4.3 uses this property to bind its Transport Layer Security (TLS) credentials to the CVM's attestation evidence.

2.2 Confidential GPUs

A confidential GPU protects selected GPU execution and memory from untrusted host software. When connected to a CVM, the platform must verify the device through identity and attestation, protect CPU-to-GPU commands and data, and explicitly authorize memory access. Device assignment alone does not allow the GPU to access CVM private memory, so communication requires either a protected I/O path or staging through shared untrusted memory [3, 74, 80].

When an NVIDIA Hopper GPU is placed in Confidential Computing (CC) mode, its High Bandwidth Memory (HBM) is logically partitioned into a CPR that holds confidential data and a smaller, unprotected region used for driver-internal staging [21]. A hardware firewall blocks external PCIe and NVIDIA Link (NVLink) access to the CPR. All data exchanged between the CVM and the GPU crosses this boundary via interfaces that use staging buffers [21, 28].

Two properties of this boundary are central to the designs that follow. First, only the Copy Engines (CEs) may cross the CPR boundary: the Streaming Multiprocessors (SMs) are confined to the CPR and fault if they access unprotected memory. Second, the CE always encrypts when copying data into a non-CPR memory region: data leaving the CPR is encrypted in hardware with 256-bit AES in Galois/Counter Mode (GCM) and decrypted on the way in, under keys derived from the Security Protocol and Data Model (SPDM) session the GPU establishes with the CVM [28].

2.2.1 Command Submission

The GPU consumes work through per-channel command queues built from the push-buffer, which holds actual GPU commands, and the General-Purpose FIFO (GPFIFO), a ring buffer whose entries each point to a pushbuffer segment [87]. Outside of CC mode, both structures lie in system memory, and submission is straightforward: the driver (i) writes the methods into the pushbuffer, (ii) appends a GPFIFO entry, (iii) advances the producer index GP_PUT, and (iv) rings the channel's doorbell [87].

In CC mode, this flow breaks because the GPU cannot access the CVM's private memory, and unprotected system memory is ruled out due to integrity concerns. GP-FIFO, GP_PUT, and push buffers consequently move into the CPR. The CPU, however, cannot write into the CPR either, causing every submission to be staged: the driver encrypts the pushbuffer into an unprotected staging buffer, and a Copy Engine has to decrypt the data into the CPR, which is itself a GPU command creating a bootstrapping problem [28].

NVIDIA resolves this with two dedicated channels. A Security Processor 2 (SEC2) channel sets up a Work Launch Channel (WLC) and Launch Confirmation Indicator

Channel (LCIC). The WLC contains two static methods, `decrypt_push` and `run_push`, that have the CE decrypt staged, encrypted commands into the CPR and trigger their execution. To submit work, the driver encrypts the user pushbuffer and a small per-submission WLC push, writes them to the staging buffer, and then rings the WLC's doorbell. The LCIC then re-arms the WLC once it completes [28].

2.2.2 Copying Data

A `cudaMemcpy` in CC mode inherits the submission path above and additionally bounces the payload through unprotected system memory, since the CE can neither read from nor write to the CVM's private memory. A Device-to-Host (D2H) copy proceeds in 3 steps: (i) The driver submits the copy command as described in Section 2.2.1. (ii) The CE encrypts and authenticates the plaintext into an unprotected buffer in system memory [21, 58]. (iii) The driver decrypts and authenticates the data in software into its destination [28].

A Host-to-Device (H2D) copy mirrors these steps: the driver encrypts the payload into the staged buffer, and the CE decrypts and verifies it into the CPR.

2.2.3 Copy-Engine Key and IV Multiplexing

The Copy Engine multiplexes many channels over a small pool of Logical Copy Engines (LCEs), independently keying and sequencing each. Each channel carries its own cryptographic context: a protected on-die slot that stores the channel's Initialization Vector (IV) state, block counter, and partial authentication state [31]. Because GCM is sequential, the engine can suspend a channel mid-transfer and later resume it, sharing a single engine fairly among multiple channels [31].

In CC mode, the GPU System Processor (GSP) derives four keys per LCE, and all channels bound to the same LCE share them [28]. Channels are separated by their IVs, which each channel manages independently and increments after every encryption or decryption. The engine forms a 96-bit IV as:

$$\text{ivMask} \oplus (\text{chan_ctr} \parallel \text{msg_ctr}) \tag{1}$$

where `ivMask` is a per-key mask from the GSP key bundle, `chan_ctr` is a 64-bit channel counter, and `msg_ctr` is a 32-bit message counter that increments with each copy [28, 31].

The GPU driver configures the channel counter: when constructing a secure channel, it increments the counter, writes it into the last 64 bits of the IV (Listing A.2), and copies this initial IV into the channel allocation parameters sent to the GPU (Listing A.1). The `ivMask` is deposited by the GSP as part of the key bundle (Listing A.3).

This construction ensures that no two channels bound to the same LCE ever use the same IV, since their channel counters differ.

When the CE seals a copy, leaving the CPR, the record it emits carries the three quantities a receiver needs to open one chunk: the 96-bit IV, the ciphertext, and the 128-bit GCM authentication tag [21, 28, 31].

$$\text{record} = [\text{iv} \parallel \text{ciphertext} \parallel \text{tag}] \tag{2}$$

The IV is a deterministic value constructed above, so a receiver that mirrors the `ivMask` and replays the same `chan_ctr` and `msg_ctr` can reconstruct it locally rather than reading it from the record.

2.3 GPUDirect RDMA

Traditionally, RDMA technology has been used to directly transfer data from system memory to RDMA-enabled NICs without involving the host CPU. Unfortunately, RDMA supports only transfers from system memory, meaning that, for GPUs with their own VRAM, data must first be copied from VRAM to system memory before a direct transfer can occur.

To solve this issue, NVIDIA introduced GPUDirect RDMA, which allows PCIe devices to perform RDMA directly with the GPU's memory regions, without involving the CPU or requiring data to be copied to system memory. At a high level, GPUDirect RDMA allows third-party PCIe devices that share the same PCIe root complex as the GPU to directly access the GPU's VRAM, with no CPU involvement other than an initial configuration phase.

Deployment model. We consider a deployment model with multiple nodes connected via an Ethernet network. Each node contains a CPU connected with a GPU over PCIe.

Initialization. A running application on the CPU handles the initialization phase. First, the application allocates GPU memory (`cuMemAlloc()`) and registers the memory region as a DMA buffer (`cuMemGetHandleForAddressRange()`). Internally, the GPU driver prepares a `dma_buf` structure and returns it to the application as a file descriptor. The application then creates a Protection Domain (PD), registers the GPU allocation as an Memory Region (MR) associated with that PD, and creates a Queue Pair (QP) and Completion Queue (CQ). The MR defines the address range accessible through RDMA and yields the remote key (rkey) required for remote access; the QP accepts work requests, while the CQ reports their completion [10, 11]. To register the GPU allocation, the application passes the `dma_buf` file descriptor to the RDMA driver, which uses it to obtain the `map_dma_buf` callback registered by the GPU driver. The callback then calls

into the GPU driver to translate the virtual address into a DMA address usable by the NIC. At this point, the GPU driver must ensure the memory pages are pinned. Finally, the RDMA driver uses the DMA address to configure the RDMA engine on the NIC. This registration and pinning workflow follows the GPUDirect RDMA control path [11].

Remote connection. To establish an RDMA channel between two GPUs, one application first initializes the NIC as described above, then sends QP metadata out-of-band using a protocol such as TCP/IP to a second application, which then goes through the same initialization process and responds with its own QP metadata. This metadata includes the remote address and rkeys, allowing each application to access the remote RDMA region set up by the other application.

Data transfer. Assume an RDMA write workflow in which an application on node 1 sends data from its local GPU to a GPU on node 2. Once the connection has been established, the application on node 1 can initiate the RDMA transfer by preparing a work request (`ibv_send_wr` structure) and posting it to the RDMA driver via `ibv_post_send`. The driver then translates this work request into a hardware-specific work request, which is sent to the NIC [11].

The actual transfer occurs in three steps: first, the NIC uses peer-to-peer (P2P) PCIe to fetch data from the GPU's memory. While the data is being fetched, the NIC uses RDMA over Converged Ethernet (RoCE) to stream it to the NIC on node 2. Finally, the NIC on node 2 uses P2P PCIe to transfer the data directly to the GPU memory buffer on node 2 [11, 66].

2.3.1 PCIe Peer Transactions

For the GPU-to-GPU write described above, the sending NIC obtains the source data using P2P PCIe reads [11]. A peer read is a non-posted PCIe transaction: the NIC sends a Memory Read (MRd) request, and the GPU returns the payload in one or more Completion with Data (CplD) Transaction Layer Packets (TLPs) [46, 64]. At the receiver, the NIC writes the payload using Memory Write (MWr) TLPs, for which PCIe returns no completion TLP [46, 64]. Transfers larger than the supported TLP max payload size are split across several TLPs [64].

2.4 Trusted I/O

Trusted I/O [3, 9] technology has emerged to obviate the need to bounce data through an untrusted buffer when a PCIe device sends data to a CVM. In particular, the

TEE Device Interface Security Protocol (TDISP), SPDM, and the IDE protocols work together to enable a trusted PCIe device to send data directly to a CVM's protected memory region. SPDM authenticates device components and secures their management sessions; TDISP controls the assignment and lifecycle of a TEE Device Interface (TDI); and IDE protects PCIe traffic [23, 24, 62, 63].

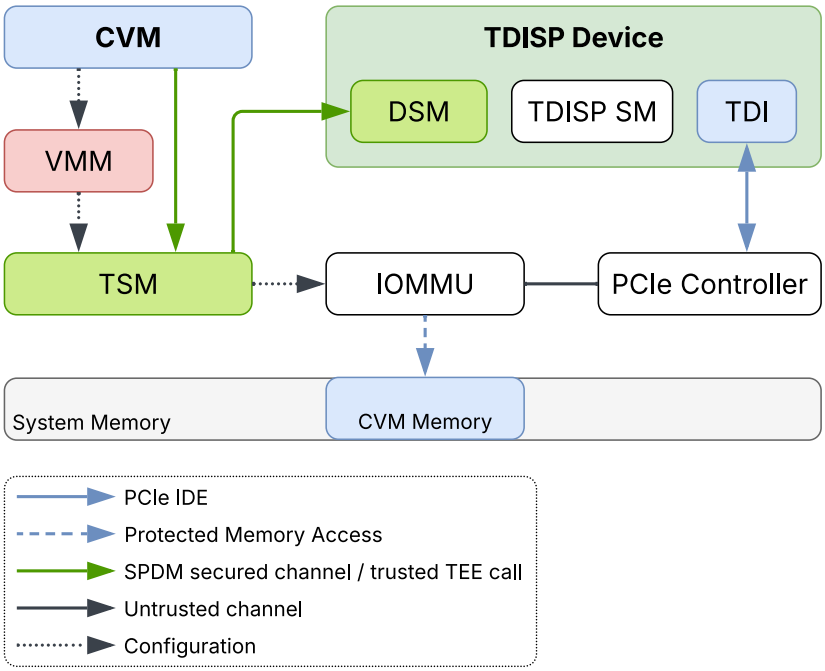


Figure 2.1: TDISP architecture

The architecture of a system providing trusted I/O is shown in Figure 2.1. Its central components are the host-side TEE Security Module (TSM) and device-side Device Security Module (DSM). The TSM coordinates device admission and protected address translation, while the DSM reports and enforces the state of its TDIs [13, 63].

A TDI is the device interface assigned to a CVM. Its identifier names the hosting PCIe Function and its Requester ID (RID), thereby identifying the origin of its DMA requests [63]. The TDI is configured before being locked and appraised via a device-interface report that includes its protected Memory-Mapped I/O (MMIO) ranges.

The TSM configures the Input/Output Memory Management Unit (IOMMU) to restrict the TDI to authorized memory. The Address Translation Service (ATS) allows the device to request and cache these translations and is required for direct P2P binding

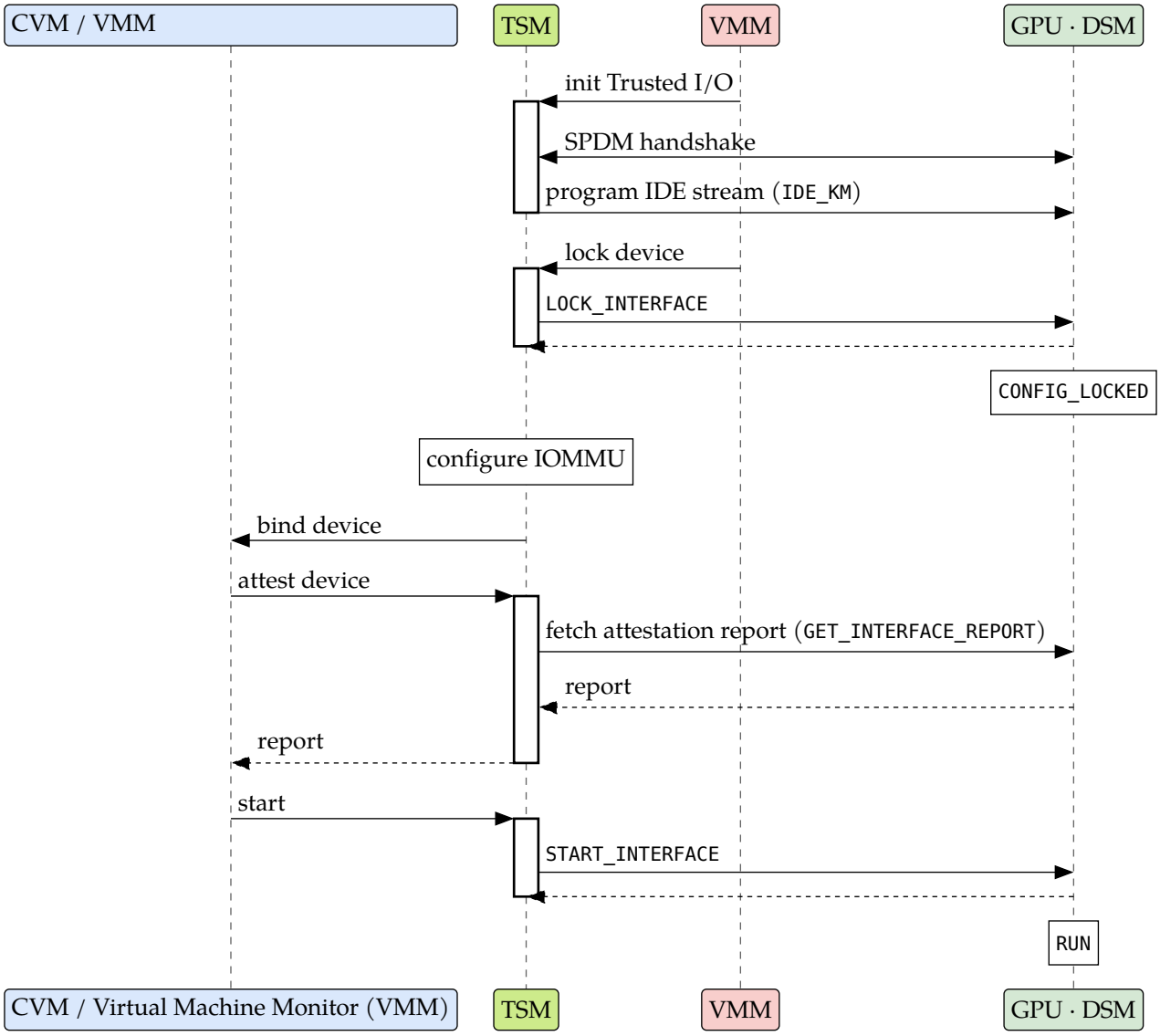


Figure 2.2: TDISP workflow

[13, 63]. For direct P2P communication, the TSM binds a running TDI to a configured selective IDE stream. IDE encrypts and authenticates the TLPs carried by this stream, ensuring confidentiality and integrity across the PCIe link [62, 63]. The complete admission workflow is shown in Figure 2.2.

2.5 IPsec and ESP Offload

IDE protects the local PCIe path but not traffic crossing the network between nodes [62]. IPsec ESP provides confidentiality, data-origin authentication, connectionless integrity, and replay protection for IP traffic [41, 73]. In transport mode, ESP protects the transport-layer payload while leaving the original IP header available for routing [41]. An ESP Security Association is unidirectional, so bidirectional communication uses one association in each direction [41, 73]. Internet Key Exchange version 2 (IKEv2) authenticates the peers and establishes and maintains these Security Associations [40]. NICs supporting full IPsec offload can apply ESP processing in NIC hardware rather than on the host CPU [1].

2.6 Modeling Communication Cost

We model every stage of a datapath with the latency-bandwidth (α, β) cost function

$$T(n) = \alpha + \frac{n}{\beta} \tag{3}$$

Here $T(n)$ is the modeled latency in seconds for transferring one payload of size n. α is the fixed per-message latency independent of the size, and β is the asymptotic bandwidth.

This model is widely used in communication performance analysis and is sufficient for our relative comparisons because, for each comparison, the deployment, topology, collective schedule, and workload are fixed while the datapath designs change the aggregate fixed latency and bottleneck throughput [32, 78, 84]. More sophisticated models, such as LogP [19] and LogGP [2], require parameters that we do not measure, so our per-transfer equation is limited to isolated transfer costs and does not model contention, queuing, or concurrent transfers.

We write a copy from memory domain x to domain y as $x \to y$, mark protected memory with p, unprotected memory with subscript u, and mark a value considered under confidential computing with a superscript C. $\hat{x}$ denotes a directly measured value. We write $x \leftarrow \hat{x}$ when a model parameter is instantiated from a measurement. The four memory domains are listed in Table 2.1.

Symbol	Memory domain
v_u	vidmem (non-CPR)
v_p	protected vidmem (CPR)
s_u	shared/unprotected sysmem
s_p	private sysmem (CVM)

Table 2.1: Notation

3 Motivation

Data movement is the key bottleneck in many cloud workloads, especially those performing machine learning inference and training [35, 61]. As such, GPUDirect RDMA [11] has been introduced to optimize inter-GPU data transfers by allowing PCIe devices, such as NICs, to access GPU memory directly without using system memory as an intermediate bounce buffer. Systems that leverage GPUDirect RDMA technology have achieved up to 69% latency reduction for send/recv operations and ~30% reduction in overall application execution time [66]. The resulting direct datapath is shown in Figure 3.1.

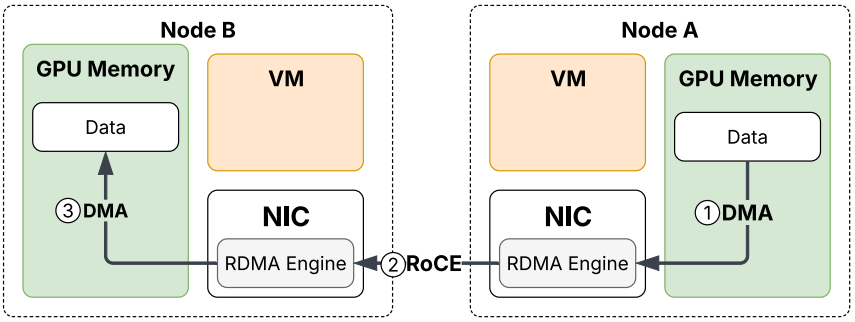


Figure 3.1: GPUDirect RDMA datapath.

The large volume of data processed in the cloud also raised questions about data privacy and integrity. To ensure those properties, confidential computing [4, 14] has emerged as the de facto standard for protecting data during execution. Recently, NVIDIA has developed confidential GPUs [15] to complement CPU-side confidential compute technologies such as AMD SEV-SNP [4] and Intel TDX [9] and to protect data processed on the GPU.

3.1 GPUDirect RDMA is Incompatible with Confidential GPUs

With these two technologies, one would expect efficient and confidential data transfers to be commonplace in the cloud. Unfortunately, the two technologies are currently incompatible because the strict security requirements of confidential computing preclude the use of GPUDirect RDMA when the GPU is operating in CC mode.

The GPU's firewall blocks external PCIe access to its compute-protected region (CPR), preventing the NIC from directly accessing the data to be transferred. Also, internal memory mappings are hidden even from the CVM, making it impossible to configure the NIC's RDMA engine [28]. Additionally, NVIDIA exposes no supported path that admits the NIC into the Trust Boundary and allows it to access the CPR. GPUDirect RDMA is explicitly unsupported under CC, and NVIDIA's Trusted I/O documentation covers the CVM-to-GPU integration rather than a direct NIC-to-GPU path [21, 28, 58].

3.2 Fallback: Bounce Buffer Through CPU Memory

Consequently, system memory must serve as a bounce buffer between the GPU and the NIC. A normal secure D2H copy, however, produces plaintext in the CVM's protected memory. This plaintext cannot be placed directly in NIC-accessible memory because the shared memory, NIC, PCIe path, and network are outside the trusted boundary defined in Section 4.2. It must first be protected under an end-to-end session key.

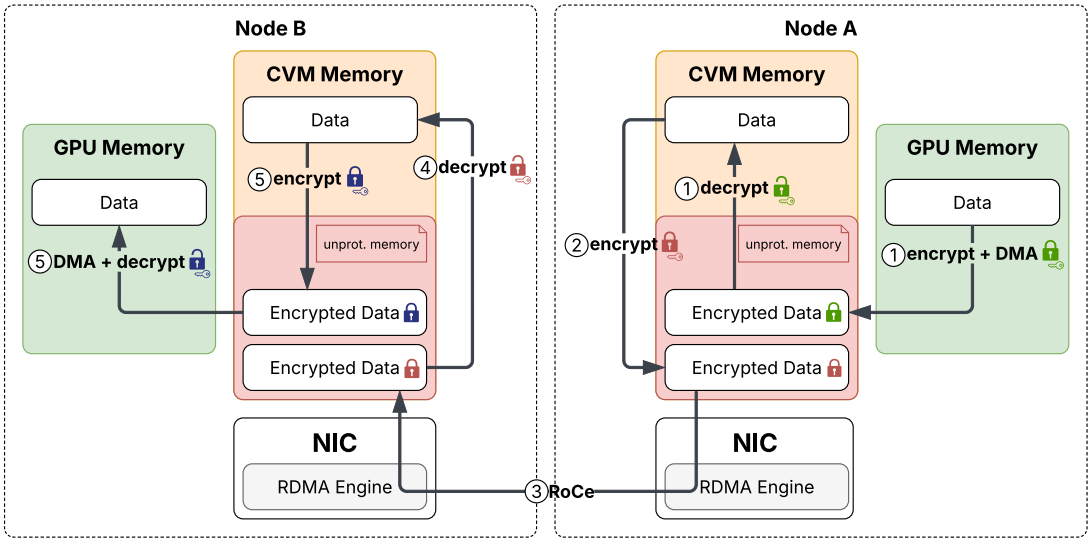


Figure 3.2: Secure CPU-bounce fallback under GPU CC.

The secure fallback is shown in Figure 3.2. It consists of five steps: (i) A D2H copy transfers the data from the sender's CPR into protected CVM memory; (ii) The sender encrypts and authenticates the data under a pre-established session key, writing the ciphertext into unprotected NIC accessible memory; (iii) The NIC transfers the data to

the receiving node using RoCE; (iv) The receiver's CVM authenticates and decrypts the ciphertext into CVM memory; (v) An H2D copy transfers the data into the receiver's CPR.

As discussed in Section 2.2.2, each D2H and H2D contributes an additional bounce through unprotected system memory. This is also reflected in Figure 3.2 placement: auto.

This fallback preserves confidentiality and integrity but imposes additional workload on the critical data path.

Problem #1: Repeated software cryptography. Each node performs two cryptographic passes in software over the payload: one as part of the secure D2H or H2D copy and another to protect the payload while it is handled by the NIC and transferred across the network. In addition, the sending and receiving Copy Engines each perform one cryptographic pass in hardware as part of the D2H and H2D copies. As illustrated in Figure 3.2, a complete end-to-end transfer, therefore, incurs six cryptographic passes. Four of those operations are performed in software on the CPU [28]. As quantified in Section 7.2, this substantially limits the throughput.

Problem #2: Staging through host memory. The fallback stages every payload through host memory at both endpoints, as shown in Figure 3.2. Compared with GPUDirect RDMA in Figure 3.1, this introduces additional copies that increase latency and memory traffic [11]. Moreover, some of these copies and their associated coordination are performed by the CPU, consuming CPU time that could otherwise be used for application computation [5].

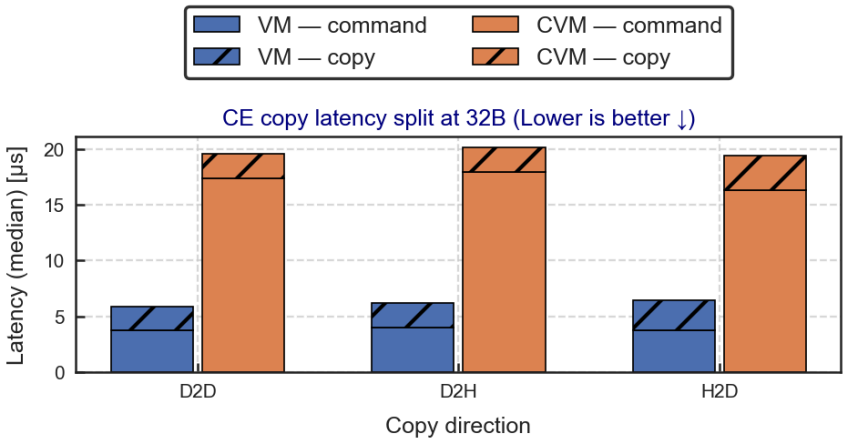


Figure 3.3: Median command-submission and Copy Engine latency of a 32 B copy.

Problem #3: Fixed transfer overhead. Secure GPU copies incur substantial fixed overhead for command submission, in addition to the time required to move and protect the payload, as quantified in Section 7.3. This overhead is paid for every independently submitted copy and therefore dominates small transfers. As shown in Figure 3.3, submitting a copy under CC takes substantially longer than executing the copy on the device.

These limitations motivate the security and performance goals presented in Chapter 4.

4 Overview

To address the limitations of the existing CPU bounce buffer fallback, we present IronLink, an architecture for secure and efficient GPU-to-GPU communication under confidential computing. We first introduce the goals and threat model guiding its design, then present its common architecture and motivate two alternative realizations.

4.1 Design Goals

We begin by presenting the properties that a robust GPU-GPU communication scheme must exhibit to leverage the advantages of both GPUDirect RDMA and confidential computing. Namely, we identify the need for data security, efficient payload protection, a direct communication channel between PCIe devices and the GPU, and the system's ability to keep the CPU off the payload path while minimizing fixed transfer overhead.

Design Goal (DG) #1: Secure end-to-end data transfers. To uphold the strong data integrity and confidentiality guarantees provided by confidential computing, an RDMA transfer solution must provide robust attestation mechanisms between the two endpoints and establish an end-to-end encrypted channel through which data can flow securely.

DG #2: Efficient device-side protection. The communication scheme should avoid redundant cryptographic transformation and perform the necessary encryption, decryption, or authentication using GPU or NIC hardware engines. Payload protection must not depend on CPU software cryptography and must not become the throughput bottleneck for the transfer.

DG #3: Direct GPU memory transfers. The data to be transferred must remain resident in GPU memory and be directly accessible to the NIC without staging through host memory. The CPU may coordinate the transfer, but must neither access nor copy the payload.

DG #4: Low overhead transfers. Fixed command-submission and coordination costs should be minimized. Where such costs are unavoidable, the design should provide mechanisms to overlap or amortize them across multiple transfers.

4.2 Threat Model

IronLink inherits and composes the trust models of three established lines of work: confidential VMs [4, 14], confidential GPUs [28], and secure RDMA transport [76].

Trusted computing base. On each node, the common TCB comprises (i) the confidential VM guest, whose memory is encrypted and integrity-protected against the untrusted hypervisor [4, 14], (ii) the confidential GPU, its Compute-Protected Region (CPR), and the engines that may access it, rooted in the GPU's hardware identity and attestation [21, 28], and (iii) the CPU TEE hardware. Host firmware, the host OS, the hypervisor, and all devices not explicitly admitted are untrusted. The exact extension of this baseline TCB depends on how payload protection is implemented and is specified in the alternatives in Section 4.4. A peer node is trusted only after mutual attestation establishes that the CVM and every device admitted to its data path are genuine and in the expected state.

Adversary capabilities. Following the confidential-computing model [4, 14, 28], the adversary controls the cloud infrastructure: the hypervisor, host operating system, firmware, and all host software outside the CVM. The adversary may spawn malicious CVMs and may attach malicious or compromised PCIe devices, including GPUs and NICs. Following the secure-RDMA model [76], the adversary additionally controls the network between nodes and may observe, drop, reorder, replay, inject, or modify any packet in transit. We further assume the adversary has physical access and can snoop the PCIe interconnect.

Assumptions and out of scope. We assume attested devices and their underlying technologies (DMA, RDMA, IDE, TDISP, SPDM) function according to their specifications, and that a device, once attested, behaves as attested [3, 9, 76]. Side-channel and denial-of-service attacks are out of scope, as are physical attacks on package internals. Availability is not guaranteed. We assume cryptographic primitives to be secure.

4.3 The IronLink Architecture

Deployment Model. We assume a homogeneous deployment in which both endpoints use the same GPU, NIC, PCIe topology, confidential-computing configuration, and software stack. IronLink is therefore deployed symmetrically at both endpoints.

On each host, software inside the CVM coordinates an attached confidential GPU and NIC. The CVM handles only the control path: it establishes trust, creates communication channels, and initiates device operations, while GPU and NIC engines protect and

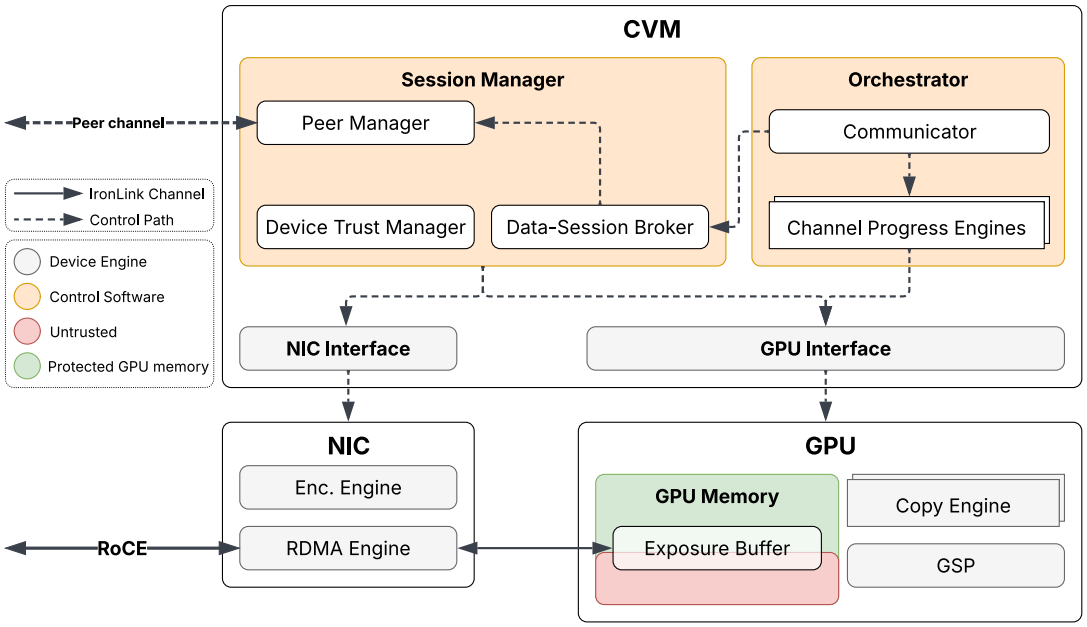


Figure 4.1: IronLink architecture overview

move the payload without staging it through CPU memory. Communication follows the lifecycle described below.

4.3.1 Trust and Session Establishment

Before payload protection and session establishment begin, the **Peer Manager** establishes the peer channel: a mutually authenticated TLS 1.3 [69] connection whose credentials are bound to each CVM's SEV-SNP attestation evidence using Remote Attestation TLS (RA-TLS) [43, 83]. The channel is the control transport between the CVMs, relaying device handshakes and communication metadata. On each host, the Device Trust Manager appraises the devices and local links required by the payload path. It makes that path available after all relevant identities, configurations, and mappings have been established and accepted. The Data Session Brokers then use the peer channel to establish the device-terminated data session under which the payload is protected.

4.3.2 IronLink Channel Creation

Once the required trust and session state are available, the Communicator creates an **IronLink Channel** (**ILC**). The IronLink Channel is the fundamental communication primitive, providing an ordered, point-to-point stream of sealed records between two

GPUs. A bidirectional link between two GPUs consists of two channels, one for each direction. The peers exchange the connection and remote-memory metadata needed by the channel over the peer channel. The Communicator then configures the required GPU and RDMA resources through the GPU and NIC Interfaces. These Interfaces are thin adapters over the existing GPU driver and verbs control paths. They also manage the channel's **Exposure Buffer**, the GPU-resident memory that the NIC accesses during transfers.

4.3.3 Record Transfer

For each submitted record, the **Channel Progress Engine** orchestrates the per-channel data transfers and ensures that required ordering guarantees are met. It communicates with the GPU Interface and NIC Interface to initiate data copies and RDMA operations. The NIC moves the record between the endpoints' GPU-resident Exposure Buffers, while the applicable GPU or NIC engines provide payload protection.

4.3.4 Invalidation and Re-establishment

Loss of required trust or data-session state invalidates the dependent channels and prevents further submissions. Communication can resume only after the affected state has been established again.

4.4 Design Alternatives

Whether the NIC can be admitted into the trust boundary depends on the available hardware and the desired TCB. IronLink therefore provides two realizations that share the architecture above but differ in whether the NIC joins the TCB and where the data session terminates.

Design 1: In-GPU Bounce. Design 1 keeps the NIC outside the TCB and terminates the data session in the GPUs. The sending Copy Engine seals each record from the CPR into a ciphertext Exposure Buffer in unprotected vidmem; the NIC transfers it via standard GPUDirect RDMA; and the receiving Copy Engine unseals it. This requires neither a trusted NIC nor a protected GPU-to-NIC path, but moves every record at both endpoints, adding copy and command-submission overhead.

Design 2: Trusted NIC. Design 2 admits an attested NIC into the TCB and terminates network protection there. The NIC accesses a plaintext Exposure Buffer directly within the CPR; IDE protects the GPU-to-NIC path, while NIC-side ESP protects the transfer between hosts. This eliminates Design 1's intermediate copies, but expands the TCB,

requires emerging GPU and NIC firmware and driver support, and subjects every record to both IDE and ESP cryptographic processing.

We present the two designs in detail in Chapter 5 and Chapter 6.

5 Design 1: In-GPU Bounce

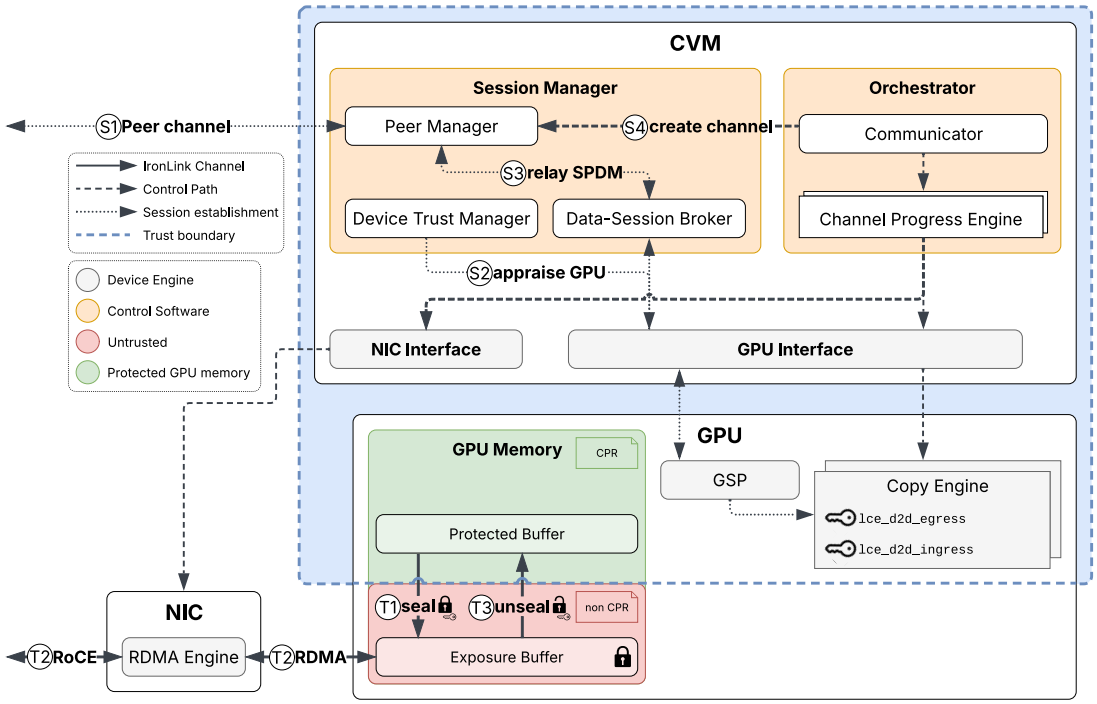


Figure 5.1: IronLink Design 1 architecture overview

Figure 5.1 provides an overview of Design 1. The Peer Managers first authenticate the two CVMs and establish their control channel (S1). On each host, the Device Trust Manager (DTM) appraises the local GPU and establishes its existing secure channel with the CVM (S2). The Data Session Brokers then relay an SPDM handshake over the peer channel to establish the GPU-terminated data session (S3), providing secure end-to-end transfers while keeping the NIC outside the trust boundary (DG #1). Once these configurations are complete, the Orchestrator creates the communication channel and binds it to a Copy Engine and Reliable Connection (RC) QP (S4).

During a transfer, the sending Copy Engine seals each record from the CPR into a ciphertext Exposure Buffer in unprotected vidmem (T1), performing payload protection in GPU hardware (DG #2). The NIC transfers the sealed record into the remote

Exposure Buffer using standard GPUDirect RDMA (T2), after which the receiving Copy Engine verifies and unseals it into the CPR (T3). Because both Exposure Buffers reside in GPU memory, the path avoids host-memory staging and CPU payload processing (DG #3). Each record, nevertheless, requires a confidential Copy Engine submission at both endpoints, so Design 1 only partially meets DG #4.

5.1 Session Manager

Device Trust Manager. Design 1 requires no additional mechanism for establishing local device trust. The Device Trust Manager invokes the confidential GPU's existing attestation and secure-channel establishment to bind the selected GPU to the CVM's trust boundary [28].

Data Session Broker. The Data Session Broker can create a data session in one of two ways. With dedicated pair sessions, each pair of communicating GPUs runs its own SPDM handshake, relayed by the Brokers over the peer channel, and the handshake terminates in the GPU firmware. With a shared key bundle, no pair session exists: the CVMs agree on key material over the peer channel and install the same bundle in their local CVM-to-GPU sessions, so a single bundle protects both CVM-to-GPU and GPU-to-GPU communication.

We choose dedicated pair sessions for three reasons: (i) Keys never enter the CVM software. Under a shared bundle, the keys exist in the CVMs, which must transport and install them; (ii) Every key has exactly one encryptor: GCM forbids reusing an IV under a key [25], and per-pair, per-engine keys keep counter management inside one engine, whereas a bundle shared across machines must coordinate a single counter space across all devices. (iii) A compromised session affects exactly one GPU pair and is rekeyed per pair, whereas a shared bundle makes any leak global.

Session establishment. The Broker initiates the pair handshake and relays its messages over the peer channel. The session terminates in the GPU System Processor that hosts an SPDM responder, yielding a shared master secret that serves as the foundation for deriving the session keys that protect the data transmitted between the two GPUs [28].

Key management. From each session's master secret, the GSP firmware derives the per LCE keys: one key per direction per LCE, `lce{x}_d2d_egress`, `lce{x}_d2d_ingress`, mirrored so that one side's egress key is the other's ingress key for the same engine. It also generates a per-key IV mask. Keys and IV masks are per-GPU pair and shared by all channels bound to that LCE (Section 2.2.3). A channel selects among them by its LCE binding and is separated from channels operating on the same keys only by its IV.

Failure handling. Integrity rests on the Authenticated Encryption with Associated Data (AEAD) tag. On tag failure, we fail closed: The plaintext is never released, and the pair data session together with all channels depending on the affected key bundle is invalidated and re-established before communication resumes. Revocation of either party's attestation evidence is handled identically, invalidating all derived keys.

5.2 Orchestrator

5.2.1 IronLink Channel

A record carried by the ILC is the Copy Engine's sealed output for one plaintext chunk P of m bytes, but with the IV dropped from the layout in Equation (2):

$$\text{record} = [\text{ciphertext} \parallel \text{tag}], \quad |\,\text{record}\,| = m + \tau \tag{4}$$

where τ is the GCM tag length ($\tau = 16$ bytes). GCM leaves the ciphertext at the length of the m-byte plaintext chunk, so each record occupies $m + \tau$ bytes.

The IV can be dropped because both LCEs on the two hosts track the message counter independently, and the IV mask and the channel counter are made to agree: the IV mask is mirrored by the key derivation, and the channel counter is a common value negotiated at channel creation (Section 5.2.2).

5.2.2 Channel Creation

Before creating a channel, we assume that the Session Manager has established a session between the peers. This session is reused by every channel created between the two peers. A channel is identified by a per-channel index k that serves both as its Copy Engine channel counter (`chan_ctr`, Section 2.2.3) and its Logical Copy Engine selector: the channel is placed on LCE number $\ell = k \bmod n_L$ for a common LCE count n_L.

As the channel counter, k must not repeat under a given key. Because the channel is unidirectional and the keys are as well, the receiver allocates k from a local monotonic counter upon channel creation. The sender uses k as its egress channel counter, so both ends share the same value and their IVs match (Section 5.2.1).

Channel creation is a single request-response over the peer channel. The sender's request includes the QP connection parameters as well as the address and `rkey` of the credit counter. The receiver replies with (k), its QP connection parameters, and the base address and `rkey` of its exposure ring. Each endpoint then binds the channel to LCE ℓ, configures its QP, and makes the channel available to the Channel Progress Engine.

5.2.3 Communicator

The Communicator multiplexes channels onto an LCE, so several IronLink Channels may share one engine. The Communicator is also responsible for channel creation. After the channel creation handshake (Section 5.2.2), each channel is bound to the corresponding engine $\ell = k \bmod n_L$ and to a single Queue Pair (QP) using the respective GPU/NIC Interface.

Even when several of these channels run in the same direction and therefore share an LCE key, they cannot collide on an IV: each channel's index k is used directly as its Copy Engine channel counter `chan_ctr` (Section 2.2.3), so distinct channels carry distinct `chan_ctr` values and the IV construction $\mathrm{ivMask} \oplus (\mathrm{chan_ctr} \parallel \mathrm{msg_ctr})$ maps each to a disjoint region of the IV space.

Per channel, the Communicator holds a channel-table entry containing the LCE and QP to which it is bound, its local exposure ring, and the remote memory targets required for payload writes and credit updates. At the sender, the entry also stores the latest receiver credit.

ℓ	engine index $\ell = k \bmod n_L$
qp	queue pair
d	ring depth (slots)
$R(h, t)$	local exposure ring in v_u: head h, tail t
$W(a, \rho)$	peer's exposure-ring target: address a, rkey ρ
M	latest receiver credit

Figure 5.2: Sender-side channel-table entry c

5.2.4 Channel Progress Engine

The Channel Progress Engine manages a channel's pipelined transfers. It runs on a CPU proxy thread that posts work and processes completions. For each send, the caller supplies a ready protected source buffer P, and for each receive, it supplies a protected destination buffer D. Synchronization with GPU kernels that produce or consume these buffers is outside of our scope.

At each endpoint, sealed records pass through a FIFO Ring R with d slots in the unprotected vidmem v_u, which serves as the exposure buffer. The ring uses monotonically increasing head and tail counters (t, h). Counter x names slot $R[x \bmod d]$. We leave this reduction implicit and index the ring directly by the counter.

Sending. At the sender, h counts records posted for transmission and t counts completed writes. The sender may reuse a local exposure slot while $h - t < d$. The receiver

periodically returns its consumed tail as the credit M. The sender may write to the receiver while $h - M < d$, preventing it from overwriting an unconsumed remote slot.

To send P, the proxy seals it into $R[h]$ and posts a signaled RDMA write with immediate to the corresponding slot in the peer's ring W. `seal` is synchronous, so the write cannot overtake the Copy Engine operation.

Algorithm 5.1: Sending on channel c

1: **procedure** SHIP(P)
2: **wait** $h - t < d \wedge h - M < d$
3: $i \leftarrow h$
4: $R[i] \leftarrow \mathbf{seal}(P)$
5: $\mathbf{writeImm}(W[i] \Leftarrow R[i])$
6: $h \leftarrow h + 1$
7: $t \leftarrow t + 1$ ▷ on Completion Queue Entry (CQE)
8: **end**

Receiving. At the receiver, h counts the number of arrived records, and t counts the number of records that have been unsealed. An RDMA write with immediate places the next record in $R[h]$ and raises a completion on the channel's CQ. The proxy advances h and unseals $R[t]$ into the destination D. It then advances t and returns it to the sender as the new credit M.

Algorithm 5.2: Receiving on channel c

1: **procedure** DELIVER(D)
2: **wait** receive CQE
3: $h \leftarrow h + 1$
4: D $\leftarrow \mathbf{unseal}(R[t])$
5: $t \leftarrow t + 1$
6: $\mathbf{writeImm}(M \Leftarrow t)$
7: **end**

The ordering and completion guarantees assumed by the algorithms are provided by the NIC Interface (Section 5.3). The credit M requires no integrity protection: a stale or smaller value only stalls the sender, whereas a larger forged value may overwrite an unconsumed slot. The resulting IV mismatch causes `unseal` to fail without releasing plaintext.

After posting the write for record i, the sender may seal record $i + 1$, while the receiver may unseal record $i - 1$ as the NIC transfers record i. The ring therefore pipelines sealing, transporting, and unsealing across consecutive records.

5.3 GPU and NIC Interfaces

GPU Interface. The GPU Interface is unchanged from Section 4.3, except for one addition. For each IronLink Channel, it opens a GPU channel bound to the same engine ℓ and allocates its exposure ring R in non-CPR. `seal` and `unseal` submit a Copy Engine copy on this GPU channel through the standard WLC/LCIC path (Section 2.2.1). They block on the copy's tracking semaphore, which the Copy Engine increments on completion, and which the driver polls in host memory [28, 87]. This makes `seal` and `unseal` synchronous.

NIC Interface. The NIC Interface is mostly unchanged from Section 4.3, with two changes: First, because each LCE advances its IV in lockstep and cannot decrypt out of order [28], records must arrive in order. We therefore use an RC Queue Pair over RoCE [10], whose transport service delivers reliably and in order [29, 42]. Second, each transfer is posted as a signaled RDMA write with immediate [57]. Each signaled write with immediate produces a receive CQE for arrival and a send CQE. Since RC preserves order and the QP identifies the channel, the receiver associates the arrival with $R[h]$ and the immediate values carry no information.

5.4 Datapath and Cost Model

A transfer contains three stages: (i) the sender seals the record from protected into unprotected vidmem $v_p \to v_u$; (ii) the NIC transfers the sealed record from v_u into the receiver's unprotected vidmem v_u; and (iii) the receiver unseals the record into protected vidmem $v_u \to v_p$.

We model the sender, transport, and receiver stages separately. The seal and unseal fixed costs include their command and Copy Engine latency:

$$T_s(n) = \alpha^C_{v_p \to v_u} + \frac{n}{\beta^C_{v_p \to v_u}}, \quad \alpha^C_{\text{seal}} := \alpha^C_{v_p \to v_u}, \quad \beta^C_{\text{seal}} := \beta^C_{v_p \to v_u} \tag{5}$$

The receiver performs the inverse operation:

$$T_r(n) = \alpha^C_{v_u \to v_p} + \frac{n}{\beta^C_{v_u \to v_p}}, \quad \alpha^C_{\text{unseal}} := \alpha^C_{v_u \to v_p}, \quad \beta^C_{\text{unseal}} := \beta^C_{v_u \to v_p} \tag{6}$$

The over-the-wire transport is standard GPUDirect RDMA, modeled as:

$$T_{\text{gdr}(n)} = \alpha_{\text{gdr}} + \frac{n}{\beta_{\text{gdr}}} \tag{7}$$

The sender's seal must be completed before the NIC can transmit a record, and the record must arrive before the receiver can unseal it. Their fixed costs, therefore, add:

$$\alpha_{\mathrm{D1}} = \alpha^{C}_{\mathrm{seal}} + \alpha_{\mathrm{gdr}} + \alpha^{C}_{\mathrm{unseal}} \tag{8}$$

The ring pipelines seal, transport, and unseal across consecutive records. Consequently, the asymptotic throughput is determined by the slowest stage as defined in Section 2.6:

$$\beta_{\mathrm{D1}} = \min\left(\beta^{C}_{\mathrm{seal}}, \beta_{\mathrm{gdr}}, \beta^{C}_{\mathrm{unseal}}\right) \tag{9}$$

Under the homogeneous deployment, we assume that sealing at the sender and unsealing at the receiver have equal latency and bandwidth:

$$\alpha^{C}_{\mathrm{unseal}} = \alpha^{C}_{\mathrm{seal}}, \quad \beta^{C}_{\mathrm{unseal}} = \beta^{C}_{\mathrm{seal}}. \tag{10}$$

Therefore, the model reduces to

$$\alpha_{\mathrm{D1}} = 2\alpha^{C}_{\mathrm{seal}} + \alpha_{\mathrm{gdr}} \quad \beta_{\mathrm{D1}} = \min\left(\beta^{C}_{\mathrm{seal}}, \beta_{\mathrm{gdr}}\right) \tag{11}$$

5.4.1 Bounding Copy Engine bandwidth

If the Copy Engine seal bandwidth is lower than the GPUDirect bandwidth, the Copy Engine limits Design 1. Otherwise, it is network-bound. Consequently, we do not require an exact value for $\beta^{C}_{\mathrm{seal}}$: it is sufficient to determine if it is smaller or larger than β_{gdr}.

Because our Testbed (Chapter 7) prevents us from directly measuring $\beta^{C}_{\mathrm{seal}} = \beta^{C}_{v_p \to v_u}$, we use the measurable copy from protected vidmem to unprotected system memory as a lower bound. Both operations read from protected vidmem and pass through the Copy Engines encryption path. Copying to system memory additionally traverses PCIe. We therefore assume that changing the destination to unprotected vidmem does not reduce the bandwidth and therefore:

$$\beta^{C}_{\mathrm{seal}} \geq \hat{\beta}^{C}_{v_p \to s_u} \tag{12}$$

Combining with the reduced model gives:

$$\hat{\beta}^{C}_{v_p \to s_u} \geq \beta_{\mathrm{gdr}} \Rightarrow \beta^{C}_{\mathrm{seal}} \geq \beta_{\mathrm{gdr}} \Rightarrow \beta_{\mathrm{D1}} = \beta_{\mathrm{gdr}} \tag{13}$$

6 Design 2: Trusted NIC

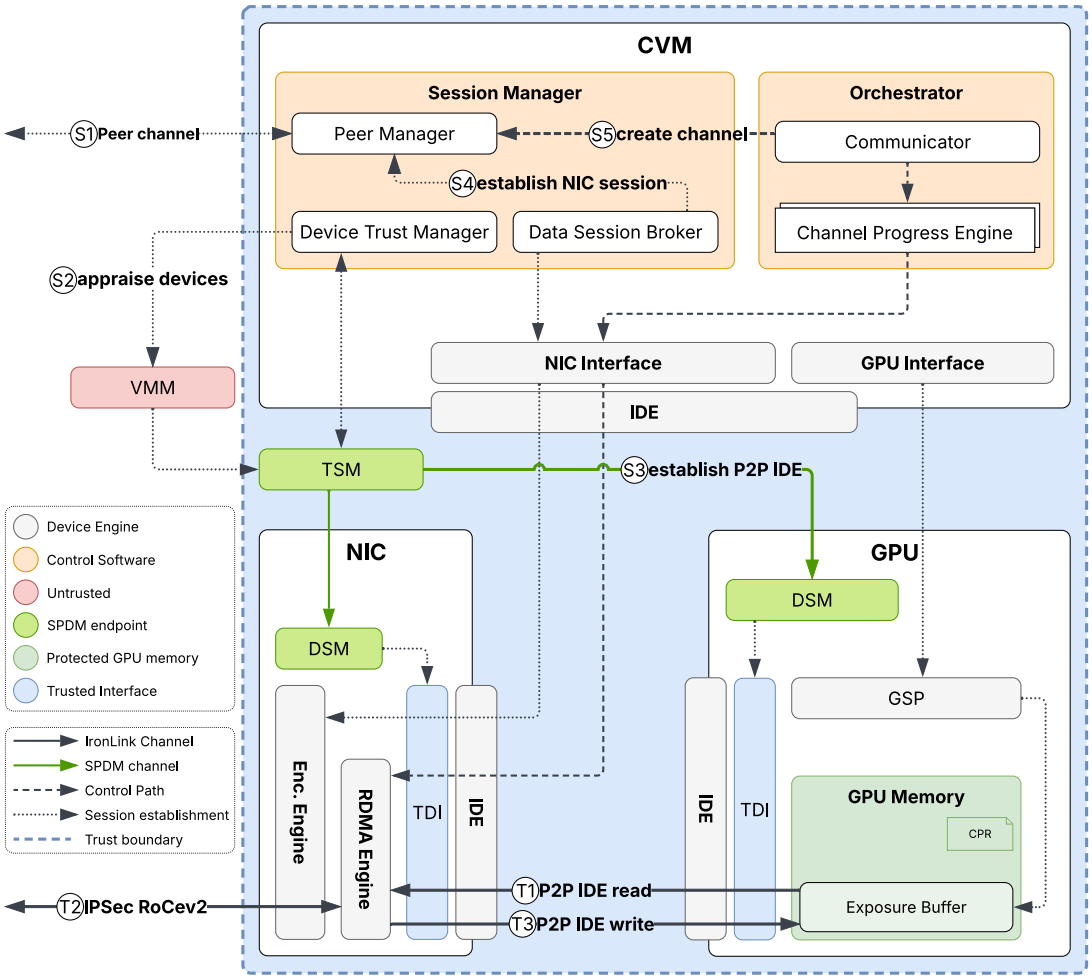


Figure 6.1: IronLink Design 2 architecture overview

Figure 6.1 provides an overview of Design 2. The Peer Managers first authenticate the two CVMs and establish their control channel (S1). On each host, the DTM appraises the local GPU and NIC (S2) and establishes a protected GPU-to-NIC path (S3). The

Data Session Brokers then use the peer channel to establish the NIC-terminated, authenticated, and encrypted session that protects the network path (S4). Together, the attested devices, protected local paths, and data sessions provide secure end-to-end transfers (DG #1). Once these configurations are complete, the Orchestrator creates the communication channel (S5).

Design 2 extends the trust boundary to the NIC and permits it to access a restricted region of protected GPU memory: the Exposure Buffer. During a transfer, the sending NIC reads plaintext directly from this buffer (T1) and authenticates and encrypts it before transmission (T2). The receiving NIC verifies and decrypts the record before writing it into the remote Exposure Buffer (T3).

All payload protection is performed in GPU, PCIe, or NIC hardware rather than CPU software (DG #2). Direct access to the CPR avoids host-memory staging and eliminates Design 1's intra-GPU bounce (DG #3). Removing the confidential copy also eliminates its command-submission latency, leaving only the standard GPUDirect RDMA control costs (DG #4).

6.1 Session Manager

6.1.1 Device Trust Manager

The DTM runs within the CVM and uses the platform's confidential I/O guest interface. Figures 6.2 and 6.3 show the step-by-step local P2P IDE path establishment sequence. Host-managed operations, such as programming an IDE stream, transitioning a TDI, or binding a TDI, are requested through the VMM, which invokes the corresponding TSM operation and relays TSM-generated secured messages through the devices.

The establishment consists of 6 steps: Steps 1 to 2 admit each device individually and are the standard TEE-IO flow. Steps 3 to 6 compose the two admitted devices into the protected GPU-to-NIC path.

Step 1: Establish SPDM sessions. The DTM initiates a secure SPDM session between the platform TSM and the NIC DSM and a separate session between the TSM and the GPU DSM. TDISP assigns the SPDM Requester role to the TSM and the Responder role to each DSM. The TSM sends the TDISP and IDE_KM management messages over these secured SPDM sessions [23, 24, 63]. Each session terminates at the corresponding DSM, which authenticates, reports, and controls its own TDI. The VMM selects and assigns one TDI per device, and the DTM receives the resulting handles. The DTM composes these two TDIs.

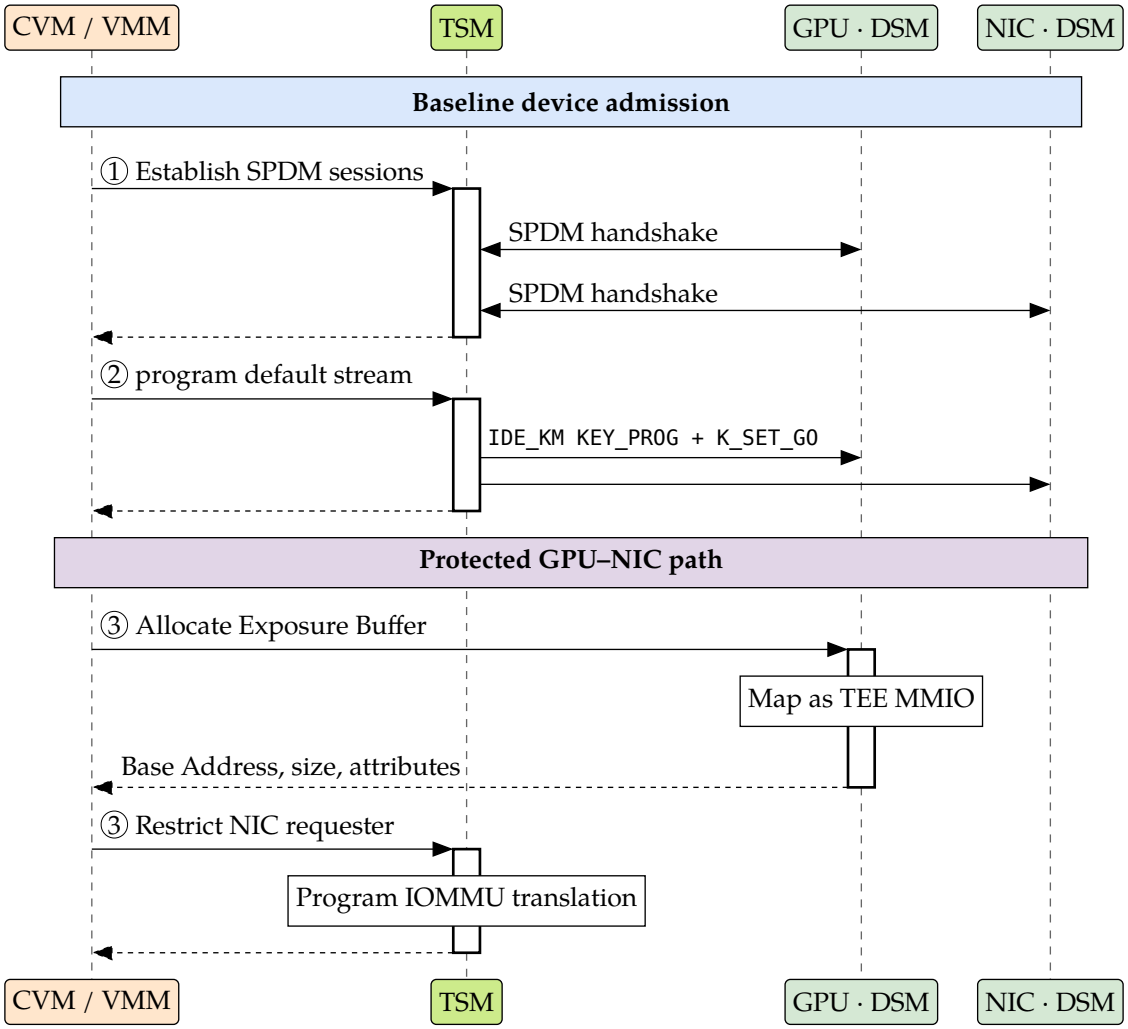


Figure 6.2: Steps 1–3

Step 2: Establish the default IDE streams. All DMA between the CVM and a device travels over an IDE stream. For each device, the TSM programs this default stream at both partner ports and keys it with `IDE_KM` [62].

Step 3: Prepare resources and translation. While the GPU TDI is in `CONFIG_UNLOCKED`, the GPU Interface allocates an Exposure Buffer in the CPR and, through a firmware operation, maps it as a protected MMIO range associated with the GPU TDI for subsequent locking and appraisal in Step 5. Because a TDI in `CONFIG_UNLOCKED` is still host-managed, the GPU interface requests this firmware operation through the VMM. The TSM then installs a protected address-translation state, realized by the platform

IOMMU, that restricts the NIC requester to the Exposure Buffer [63]. As with the Exposure Buffer, the VMM proposes this configuration and the DTM accepts it only after the appraisal in Step 5.

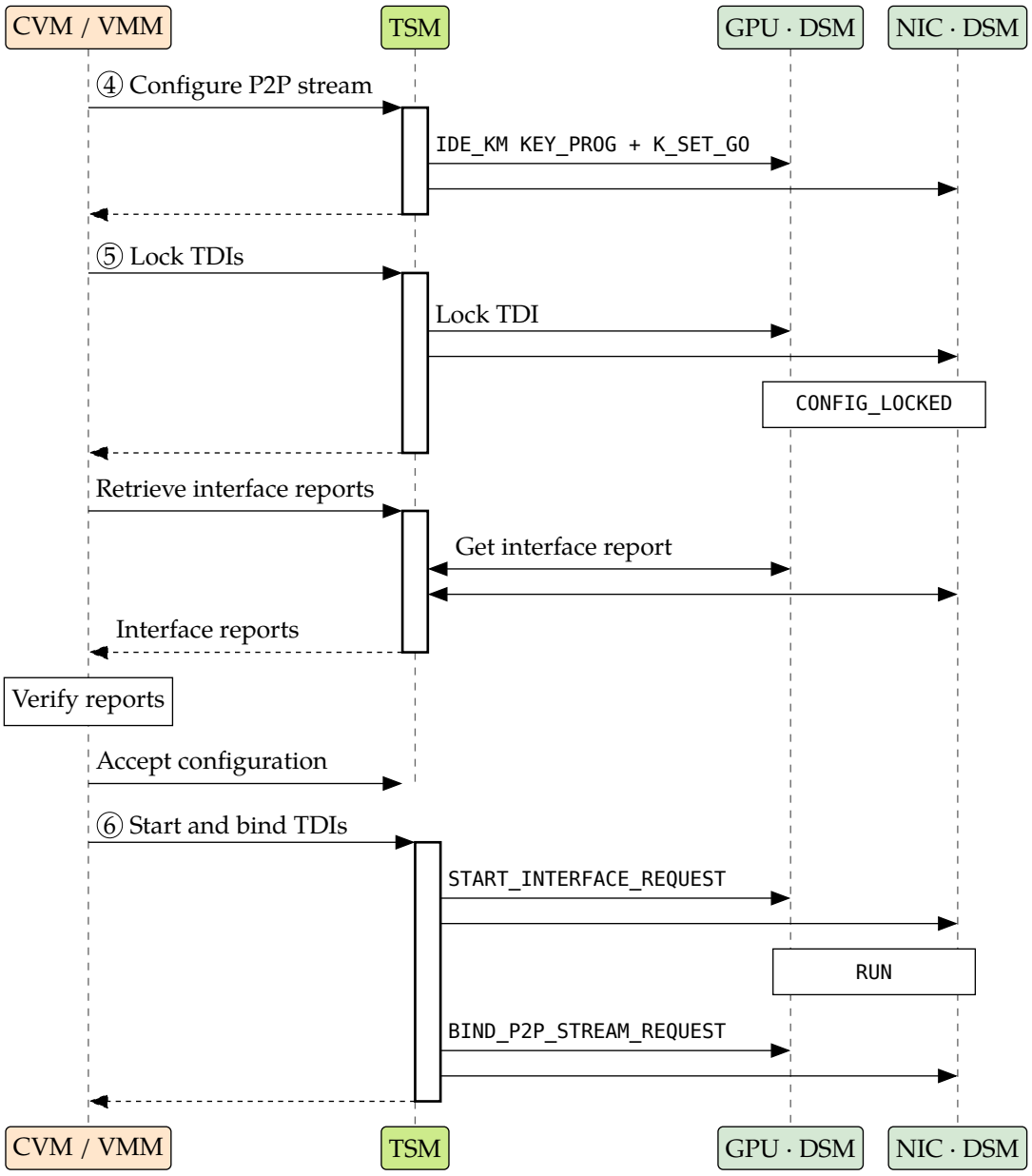


Figure 6.3: Steps 4–6

Step 4: Configure the P2P IDE stream. The TSM programs a selective IDE stream at both partner ports using the same Stream ID and algorithm at each endpoint and configuring the associations required by IDE. Because this association can be broader than the Exposure Buffer, it only selects the stream; the GPU TDI must still enforce the exact protected MMIO range [62, 63]. At the NIC port, the address association covers the Exposure Buffer, causing translated Memory Requests targeting that range to select the P2P stream. An intervening PCIe switch may forward the Selective IDE stream without terminating its protection or obtaining its keys [62].

The TSM then provisions the stream using IDE_KM over the respective SPDM session. It uses `KEY_PROG` to install matching AES-GCM keys and IV state in the transmitting and corresponding receiving port of each IDE substream. Each traffic direction and substream use separate key material [62]. Finally, the TSM uses `K_SET_GO` to trigger the IDE at the receiver's ports [62]. The configured and keyed P2P IDE stream is then ready to be bound to the GPU and NIC TDIs in Step 6.

Step 5: Lock and appraise TDIs. The TSM issues a `LOCK_INTERFACE_REQUEST` for each TDI with direct P2P permitted, after which the platform assigns both locked TDIs to the CVM. The TSM retrieves each `DEVICE_INTERFACE_REPORT`, and the DTM appraises the device identities and measurements and checks that the reports match the prepared configuration. In particular, the NIC report must correspond to the selected NIC TDI, while the GPU report must describe the Exposure Buffer as the expected protected MMIO range. If all checks pass, the DTM signals acceptance of the configuration to the TSM.

Step 6: Start and bind TDIs. After acceptance, the TSM issues a `START_INTERFACE_REQUEST` for each TDI, transitioning it to `RUN`. The TSM then issues a `BIND_P2P_STREAM_REQUEST` independently for each TDI. Binding requires ATS and a correctly configured and keyed P2P IDE stream. The NIC obtains its Exposure Buffer translation over the default IDE stream. Direct memory accesses targeting a translated address in the Exposure Buffer then use the P2P stream, while unmatched or untranslated traffic follows the default stream rules [63].

GPU–NIC path validity. The DTM makes the path available for channel creation only after all preceding steps have succeeded. The path and all channels created from it form a single fail-closed dependency chain: they remain usable only while both TDIs, the Exposure Buffer mapping, the protected translation state, and the P2P IDE stream remain valid. If the establishment fails, no path is made available, and completed operations are unwound. If any dependency later fails, the path becomes unusable, dependent channels are torn down, and completed operations are unwound using the revocation sequence described below.

Revocation. Revocation invalidates the protected GPU-to-NIC path, causing dependent channels to block new work, drain, or abort their QPs and revoke their memory registrations. The TSM then issues an `UNBIND_P2P_STREAM_REQUEST` for each TDI. It then retires the P2P stream keys [62, 63]. Both TDIs stay in `RUN` and both default streams stay up, so the devices remain admitted and usable by the CVM. The GPU Interface then removes the TEE-MMIO mapping and frees the Exposure Buffer. A device reset, security-relevant configuration change, or a bound IDE stream entering `Insecure` invalidates the path immediately and drives the affected TDI toward `ERROR` [62, 63]. Resumption requires that the path be appraised and reestablished.

6.1.2 Data Session Broker

Building on the IPsec mechanisms described in Section 2.5, the Data Session Broker establishes a NIC-terminated data session that protects RoCEv2 traffic between the two NICs. It consists of an ESP Security Association in each direction [41, 73]. Transport-mode ESP encrypts and authenticates the User Datagram Protocol (UDP) and RoCEv2 payload and provides replay protection while the IP header remains available for routing [41, 73].

After the Protected GPU-to-NIC path is available, the Brokers run IKEv2 between the CVMs and accept only an IKE identity corresponding to the remote CVM [40]. The Brokers also restrict the negotiated traffic selectors to RoCEv2 traffic between the admitted NIC endpoints. The NIC Interfaces then install the resulting ESP Security Associations and policies into the admitted NIC functions for full packet offload [1].

The data session becomes available only after both peers confirm that the ESP state is installed. Expiration of either association, loss of offload, or failure of either Protected GPU-to-NIC path invalidates the data session and all dependent channels.

6.2 Orchestrator

IronLink Channel. Design 2 retains the ordered, unidirectional IronLink Channel abstraction of Design 1 (Section 5.2.1). Unlike Design 1, a record is the m-byte plaintext chunk itself. It resides in the protected Exposure Buffer.

Communicator. The Communicator reuses Design 1's request-response channel creation and channel table organization (Section 5.2.3). Channel creation begins only after the protected GPU-to-NIC path and data session are available. At each endpoint, the Communicator assigns the channel a d-slot slice of the Exposure Buffer, registers it as an MR on the admitted NIC function, and creates an RC QP and CQ. The peers exchange the same QP, remote-ring, and credit metadata as in Design 1. No Copy

Engine channel, LCE binding, or `chan_ctr` is required. Multiple channels may reuse the same data session while retaining separate buffer slices.

Channel Progress Engine. The Channel Progress Engine reuses Design 1's ring counters, receiver credits, and completion handling (Section 5.2.4), but submits no GPU operations. For a send, the caller supplies a ready buffer in the local exposure ring, and the proxy posts a signaled RDMA write with immediate directly from $R[h]$ to $W[h]$. At the receiver, the arrival CQE makes $R[h]$ available directly to the caller without unsealing.

6.3 GPU and NIC Interfaces

GPU Interface. The GPU Interface provides the device-specific operations to provision or select the Exposure Buffer and to associate its exact range with the selected GPU TDI as TEE MMIO. It returns the range's base, size, ID, and attributes so that the Device Trust Manager can verify this mapping in the `DEVICE_INTERFACE_REPORT` [63]. The Interface removes the association during teardown.

NIC Interface. The NIC Interface resolves the selected TDI's Function ID to the RDMA context that originates GPU-directed DMA and creates the channel's MR, QP, and CQ on this Function. For the Data Session Broker, it installs the negotiated ESP Security Associations and traffic policy for full-packet offload and selects RoCEv2 endpoints covered by that policy [73].

6.4 Datapath and Cost Model

A transfer uses the PCIe transaction structure described in Section 2.3.1 and consists of three stages: (i) the sender's NIC reads the payload from the sender's GPU using an IDE-protected peer read; (ii) the NICs exchange ESP-protected RoCEv2 packets; and (iii) the receiver NIC writes the payload using an IDE-protected memory write. The unprotected GPUDirect transport is modeled as:

$$T_{\mathrm{gdr}(n)} = \alpha_{\mathrm{gdr}} + \frac{n}{\beta_{\mathrm{gdr}}} \tag{14}$$

Its latency can be decomposed into

$$\alpha_{\mathrm{gdr}} = \alpha_{\mathrm{cmn}} + \alpha_{\mathrm{rd}} + \alpha_{\mathrm{net}} + \alpha_{\mathrm{wr}} \tag{15}$$

where α_{cmn} is some common cost unchanged by protection. Design 2 keeps these three stages but replaces them with more expensive protected variants, giving:

$$\alpha_{\mathrm{D2}} - \alpha_{\mathrm{gdr}} = \left(\alpha_{\mathrm{rd}}^{\mathrm{IDE}} - \alpha_{\mathrm{rd}}\right) + \left(\alpha_{\mathrm{net}}^{\mathrm{ESP}} - \alpha_{\mathrm{net}}\right) + \left(\alpha_{\mathrm{wr}}^{\mathrm{IDE}} - \alpha_{\mathrm{wr}}\right) = \Delta\alpha^{\mathrm{IDE}} + \Delta\alpha^{\mathrm{ESP}} \quad (16)$$

The 3 steps can be pipelined across consecutive records, giving:

$$\beta_{\mathrm{gdr}} = \min(\beta_{\mathrm{rd}}, \beta_{\mathrm{net}}, \beta_{\mathrm{wr}}) \quad (17)$$

For Design 2, IDE imposes a bandwidth ceiling $\beta^{\mathrm{IDE}} = \min\left(\beta_{\mathrm{rd},s}^{\mathrm{IDE}}, \beta_{\mathrm{wr},r}^{\mathrm{IDE}}\right)$ on the PCIe stages, and ESP imposes a ceiling $\beta_{\mathrm{net}}^{\mathrm{ESP}}$ on the network stage. Therefore, our cost model is:

$$\begin{aligned} T_{\mathrm{D2}}(n) &= \alpha_{\mathrm{D2}} + \frac{n}{\beta_{\mathrm{D2}}} \\ \alpha_{\mathrm{D2}} &= \left(\alpha_{\mathrm{gdr}} + \Delta\alpha^{\mathrm{IDE}} + \Delta\alpha^{\mathrm{ESP}}\right) \\ \beta_{\mathrm{D2}} &= \min\left(\beta_{\mathrm{gdr}}, \beta^{\mathrm{IDE}}, \beta^{\mathrm{ESP}}\right) \end{aligned} \quad (18)$$

6.4.1 Estimating fixed latency

The ESP term contains only the additional NIC-to-NIC cost

$$\Delta\alpha^{\mathrm{ESP}} = \alpha_{N_s \to N_r}^{\mathrm{ESP}} - \alpha_{N_s \to N_r} \quad (19)$$

Here, N_s and N_r denote the sender and receiver NICs, respectively.

For $\Delta\alpha^{\mathrm{IDE}}$, we assume homogeneous IDE processing and let $\delta\alpha^{\mathrm{IDE}}$ be the fixed latency increment each operation incurs at both peers of one IDE link. The sender comprises an MRd and a CplD while the receiver comprises an MWr. Each TLP is processed once at its transmitter and once at its receiver, giving six operations:

$$\Delta\alpha^{\mathrm{IDE}} = 6\delta\alpha_{\mathrm{IDE}} \quad (20)$$

6.4.2 Estimating asymptotic bandwidth

Following J. Lawley [46], we define packet efficiency as payload divided by payload plus per-packet overhead and obtain theoretical payload throughput by multiplying that efficiency by the link bandwidth. Similarly, we also base the theoretical throughput of a pipelined PCIe read only on its completion traffic [46]. The sender-side GPU read is consequently limited by the CplD stream that the GPU returns to the requesting NIC. The receiver side consists of only a PCIe write and is therefore limited by that MWr. For $q \in \{\mathrm{CplD}, \mathrm{MWr}\}$ we define the packet efficiency

$$\eta_q^{\mathrm{IDE}} = \frac{P_q}{P_q + h_q + h^{\mathrm{IDE}}} \quad (21)$$

where P_q is the max payload size [46] of a TLP of kind q, h_q^{plain} the ordinary non-payload bytes and h^{IDE} the additional bytes introduced by IDE. With $\beta_{\text{pcie}}^{\text{line}}$ being the theoretical PCIe line rate, the simplified IDE ceiling is

$$\beta^{\text{IDE}} = \beta_{\text{pcie}}^{\text{line}} \min\left(\eta_{\text{CplD}}^{\text{IDE}}, \eta_{\text{MWr}}^{\text{IDE}}\right) \tag{22}$$

PCIe permits payloads up to 4096 B, but the maximum message size is limited by the devices on the path [46]. R. Neugebauer *et al.* [56] identify 256 B as a typical maximum message size value, so we also instantiate our model with $P_q = 256B$. Under PCIe 5.0 x16, 64-bit addressed memory, no Aggregation, no Flow Control Unit (FLIT), no End-to-End Cyclic Redundancy Check (ECRC), and Protected Cyclic Redundancy Check (PCRC) enabled, the other parameters are:

$$h^{\text{IDE}} = 4B_{\text{prefix}} + 12B_{\text{MAC}} + 4B_{\text{PCRC}} = 20B \tag{23}$$

$$h_{\text{CplD}} = 12B_{\text{header}} + 4B_{\text{SN}} + 4B_{\text{LCRC}} = 20B \tag{24}$$

$$h_{\text{MWr}} = 16B_{\text{header}} + 4B_{\text{SN}} + 4B_{\text{LCRC}} = 24B \tag{25}$$

as specified by the PCIe specifications [62, 64]. The corresponding efficiencies are:

$$\eta_{\text{CplD}}^{\text{IDE}} = \frac{256\ \text{B}}{256\ \text{B} + 20\ \text{B} + 20\ \text{B}} \approx 0.8649 \tag{26}$$

$$\eta_{\text{MWr}}^{\text{IDE}} = \frac{256\ \text{B}}{256\ \text{B} + 24\ \text{B} + 20\ \text{B}} \approx 0.8533 \tag{27}$$

At a line rate of $\beta_{\text{pcie}}^{\text{line}} = 63.02\ \text{GB/s}$ [12] the resulting IDE bandwidth ceiling is

$$\beta^{\text{IDE}} = \beta_{\text{pcie}}^{\text{line}} \min\left(\eta_{\text{CplD}}^{\text{IDE}}, \eta_{\text{MWr}}^{\text{IDE}}\right) \approx 53.8\ \text{GB/s} \tag{28}$$

7 Evaluation

We evaluate IronLink from individual datapath stages to distributed workloads: (i) stage bandwidth, (ii) stage latency, (iii) end-to-end flow completion time derived from composed cost models, (iv) collective completion time across logical algorithms and cluster sizes, and (v) communication makespan for distributed ML workloads.

We compare the GPUDirect RDMA and CPU bounce baselines with projected Designs 1 and 2. We ask how accurately the composed models predict measurable datapaths (Q1); how both designs compare with the baselines in latency, bandwidth, and flow completion time (Q2); how message size, collective algorithm, and GPU count affect their relative performance (Q3); and how these effects translate to projected distributed training time (Q4).

Section 7.4.1 addresses Q1. The component-level results in Section 7.2 and Section 7.3, together with the complete-datapath comparison in Section 7.4.2, address Q2. Section 7.5 and Section 7.6 address Q3 and Q4, respectively.

Evaluation scope. We do not build a complete IronLink system. Instead, we measure available primitives, validate the composed models where possible, and use them to project the remaining results. All complete Design 1 and Design 2 datapaths are therefore projections.

Testbed. We run our experiments on an AMD SEV-SNP-enabled machine with two AMD EPYC 9654 CPUs (96 cores each, hyperthreading disabled), 1.5 TB of DDR4 DRAM, and an NVIDIA H100 GPU with Confidential GPU (CGPU) capabilities (CUDA v13.0). The server runs NixOS 24.11 (Linux kernel v6.18.6); (C)VMs use Ubuntu 24.04 (Linux kernel v6.18.0 with patches). The client (C)VM is assigned the GPU via pass-through. The two hosts are directly connected through NVIDIA ConnectX-7 NICs over a 200 GbE link (25 GB/s nominal).

Only one endpoint has a GPU. Design 1 cannot be measured because the H100 exposes no allocatable unprotected vidmem. Design 2 cannot be measured because the installed ConnectX-7 variants support neither IDE nor the IPsec full-packet offload required to protect RoCEv2 traffic.

7.1 Methodology

7.1.1 Measurements

RDMA. We measure the RDMA path with perftest [8] over a single unidirectional QP. We use `ib_write_bw` with plain RDMA writes for bandwidth, and `ib_write_lat` with write-with-immediate for latency, since write-with-immediate produces a receiver-side completion without polling the destination buffer. GPUDirect uses the stock GPUDirect path, registering the GPU buffer with the NIC through `dma_buf` rather than the `nv_peer_mem` module. Since perftest cannot stage a transfer through host memory, we patch it to implement this functionality. Because only one endpoint has a GPU, GPUDirect is measured between the GPU memory and a system memory peer.

Plain GPU copies. We measure the Copy Engine copies that CUDA exposes (intra-GPU Device-to-Device (D2D), D2H, and H2D) with `nvbandwidth` [17], both with confidential computing off (the non-CC baselines) and on.

Sealed copies. A sealed copy copies protected vidmem to sysmem without decrypting on the host side. Unsealing is the reverse. `nvbandwidth` can measure neither because CUDA always performs the full software decrypt/encrypt and offers no way to bypass it. We therefore implement the measurement ourselves at the driver level, adapting Y. Yan *et al.* [87]. To keep the result bound to hardware rather than command submission, we coalesce all iterations into a single pushbuffer segment

$$\text{sem_ts } t_0 \parallel \text{copy} \times \text{iters} \parallel \text{sem_ts } t_1 \tag{29}$$

and submit it in one shot. For n transfers of size s the measured copy bandwidth is

$$\hat{\beta} = \frac{n_{\text{tot}}}{t_1 - t_0} \text{ with } n_{\text{tot}} = n \cdot s \tag{30}$$

Command vs Copy Latency. To split a GPU-involved copy latency into command submission and on-device copy time, we put three entries into the pushbuffer: (i) a semaphore g_{t_0}, (ii) the copy, and (iii) a semaphore g_{t_1}. We measure CPU timestamps c_{t_0} before submission and c_{t_1} after completion. From these we can calculate the copy latency $\hat{\alpha}_{\text{cpy}} = g_{t_1} - g_{t_0}$, the total latency $\hat{\alpha} = c_{t_1} - c_{t_0}$, and the command submission latency $\hat{\alpha}_{\text{cmd}} = \hat{\alpha} - \hat{\alpha}_{\text{cpy}}$. Command latency includes all measured kernel time outside Copy Engine execution, including pushbuffer construction, secure submission, and completion signaling. We get the fixed latency as the median of repeated measurements at a message size of 32 B.

7.1.2 Parameter Sources and Assumptions

We use the (α, β) model introduced in Section 2.6 and applied for Design 1 in Section 5.4 and Design 2 in Section 6.4 using the parameters measured with the methods described above.

GPUDirect RDMA. We use the measured GPUDirect (α, β) parameters for the homogeneous two-GPU deployment, assuming that replacing the system-memory peer with an otherwise identical GPU endpoint does not alter them.

CPU bounce. We measure the fallback from Section 3.2 as three independent serial stages: (i) D2H followed by software encryption, (ii) system memory RDMA and (iii) software decryption followed by H2D. The one-sided and two-sided models are

$$T^1_{\mathrm{bb}}(n) = \alpha_{\mathrm{D2H+enc}} + \alpha_{\mathrm{rdma}} + \frac{n}{\min(\beta_{\mathrm{D2H+enc}}, \beta_{\mathrm{rdma}})} \tag{31}$$

and

$$T^2_{\mathrm{bb}}(n) = \alpha_{\mathrm{D2H+enc}} + \alpha_{\mathrm{rdma}} + \alpha_{\mathrm{dec+H2D}} + \frac{n}{\min(\beta_{\mathrm{D2H+enc}}, \beta_{\mathrm{rdma}}, \beta_{\mathrm{dec+H2D}})} \tag{32}$$

We validate T^1_{bb} against the separately measured one-sided path and use T^2_{bb} for comparisons and simulation.

Design 1. We take the transport parameters directly from the GPUDirect measurements: $\alpha_{\mathrm{gdr}} \leftarrow \hat{\alpha}_{\mathrm{gdr}}, \beta_{\mathrm{gdr}} \leftarrow \hat{\beta}_{\mathrm{gdr}}$. Since the limitation described in the testbed setup prevents us from directly measuring the protected-to-unprotected vidmem copy, we use the measured Copy Engine seal into unprotected sysmem as a lower bound on its bandwidth:

$$\beta^C_{\mathrm{seal}} \geq \hat{\beta}^C_{v_p \to s_u}. \tag{33}$$

For fixed latency, we approximate both intra-GPU operations with the measured confidential D2D latency:

$$\alpha^C_{\mathrm{seal}} \approx \hat{\alpha}^C_{v_p \to v_p}, \quad \alpha^C_{\mathrm{unseal}} \approx \hat{\alpha}^C_{v_p \to v_p}. \tag{34}$$

We obtain this latency using our driver-level benchmark, so it includes the command, copy, and completion latencies. This approximation assumes that Copy Engine encryption or decryption adds little fixed latency to an otherwise equivalent intra-GPU copy.

Design 2. As in Design 1, we take the parameters α_{gdr} and β_{gdr} directly from the GPUDirect measurements. For the network latency, we use the secure minus plain write latency in the sRDMA evaluation [76], which gives the estimate

$$\Delta\alpha^{\mathrm{ESP}} \approx 11.5\,\mu\mathrm{s} - 9.5\,\mu\mathrm{s} = 2\,\mu\mathrm{s} \tag{35}$$

This may overestimate latency because sRDMA processes packets on programmable SmartNIC cores rather than on an inline FPGA-based AES-GCM engine.

No IDE-on vs. IDE-off result exists for the GPU-to-NIC path. We therefore use 150 ns of fixed additional latency of the inline AES-GCM engine of S. Ghosh *et al.* [26]. We assume that every GPU and NIC endpoint integrates a line-rate AES-GCM engine with no greater fixed latency. That engine uses AES-128, whereas PCIe IDE requires AES-GCM-256 [62]. AES-256 has 14 rounds instead of the 10 rounds used by AES-128 [77]. Scaling the complete pipeline by $\frac{14}{10}$ yields a per-operation estimate of 210 ns. Applying it independently to all six IDE operations in (20) yields:

$$\Delta\alpha^{\mathrm{IDE}} = 6\delta\alpha^{\mathrm{IDE}} = 1.26\,\mu\mathrm{s} \tag{36}$$

The Design 2 fixed latency is therefore

$$\alpha_{\mathrm{D2}} = \alpha_{\mathrm{gdr}} + \Delta\alpha^{\mathrm{IDE}} + \Delta\alpha^{\mathrm{ESP}} = \alpha_{\mathrm{gdr}} + 3.26\,\mu\mathrm{s}. \tag{37}$$

D. Iliadis-Apostolidis *et al.* [34] measure 222.78 Gbit/s for fully offloaded ESP over UDP on an NVIDIA BlueField-3, which includes a ConnectX-7 NIC [18]. Since RoCEv2 encapsulates RDMA in UDP/IP [10], we assume the same offload throughput and set:

$$\beta^{\mathrm{ESP}} = \frac{222.78\ \mathrm{Gbit/s}}{8} \approx 27.85\,\mathrm{GB/s} \tag{38}$$

Concurrent stages. The stage parameters are measured independently. When composing them into a pipelined cost model, we assume that they retain their independently measured rates and do not introduce additional resource contention.

The benchmark implementations, patches, raw results, plotting scripts, and simulation configurations used in this evaluation are available at https://github.com/TUM-DSE/gpu-network-io.

7.2 Bandwidth

We use microbenchmarks to measure the bandwidth of each datapath stage, parameterize the models, and compare their bandwidth projections against the baselines.

Figure 7.1 compares the non-CC GPUDirect ceiling with the measured one-sided CPU bounce required when the NIC cannot access protected GPU memory. The System Memory RDMA achieves the same bandwidth as GPUDirect once the network becomes the bottleneck. At the asymptotic bandwidth, the bounce exhibits 80% less

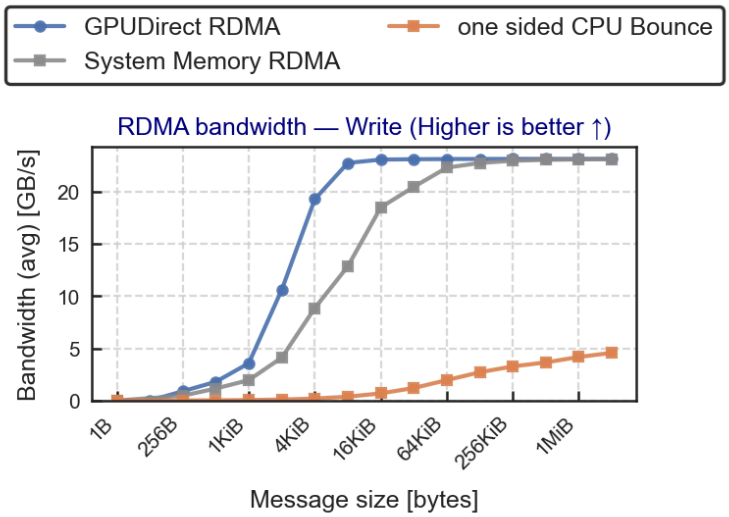


Figure 7.1: Measured write bandwidth for GPUDirect RDMA, System Memory RDMA, and the one-sided CPU bounce.

bandwidth than GPUDirect. This loss persists even after fixed costs have been amortized, indicating a per-byte bottleneck rather than a high fixed per-message latency.

The Device-to-Host bandwidth reveals this bottleneck. Compared to the non-CC Copy Engine ceiling, sealing reduces bandwidth by 32%. However, the sealed path achieves 4.02× the bandwidth of CUDA's standard confidential D2H path. As

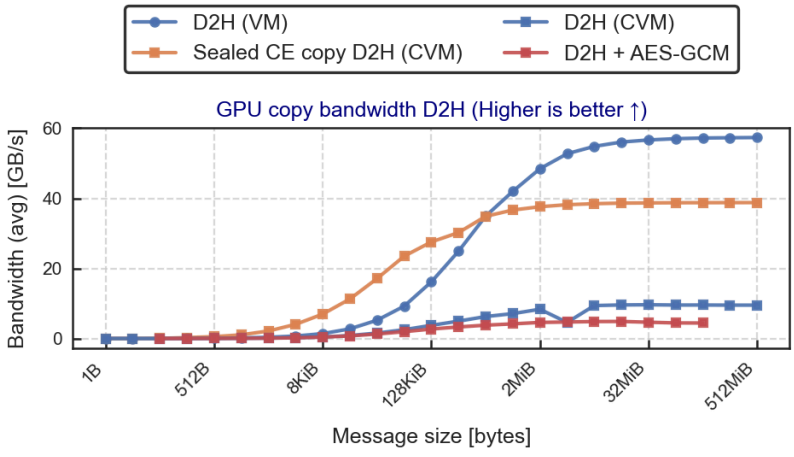


Figure 7.2: Measured D2H bandwidth for D2H (VM), Sealed CE copy D2H (CVM), D2H (CVM), and D2H + AES-GCM.

described in Section 2.2.2, both paths use the Copy Engine to encrypt the payload into unprotected system memory, but CUDA then decrypts it into protected system memory using slow software decryption. We therefore attribute most of the CPU bounce bandwidth loss to software cryptography. This agrees with prior studies on Hopper that identify software encryption as the dominant bottleneck for data movement in confidential computing [28, 75].

Parameter	Measurement	Peak average bandwidth
$\hat{\beta}_{\text{gdr}}$	GPUDirect RDMA	23.15 GB/s
$\hat{\beta}_{\text{D2H+enc}}$	D2H + AES-GCM	4.87 GB/s
$\hat{\beta}_{\text{rdma}}$	System Memory RDMA	23.12 GB/s
$\hat{\beta}_{\text{dec+H2D}}$	AES-GCM + H2D	4.53 GB/s
$\hat{\beta}^{C}_{v_p \to s_u}$	Sealed CE copy D2H (CVM)	38.78 GB/s

Table 7.1: Measured bandwidth quantities used to parametrize the cost models

Table 7.1 lists the measured bandwidth quantities used to instantiate or bound the cost models. We can also see that the sealed copy reaches 38.78 GB/s, 68% above the 23.15 GB/s GPUDirect RDMA bandwidth. Therefore the inequality (13) holds and:

$$\beta^{C}_{\text{seal}} \geq \hat{\beta}^{C}_{v_p \to s_u} = 38.78\ \text{GB/s} > 23.15\ \text{GB/s} = \hat{\beta}_{\text{gdr}} \Rightarrow \beta_{\text{D1}} = \hat{\beta}_{\text{gdr}} \tag{39}$$

This confirms the network as the slower stage in the Design 1 model.

7.3 Latency

We use microbenchmarks to measure fixed per-message latency and isolate the contribution of confidential GPU command submission.

Figure 7.3 compares the measured fixed latencies at 32 B. GPUDirect RDMA and System Memory RDMA have fixed costs of only 1.9 µs and 1.22 µs, respectively. The confidential D2H and H2D are already nearly as expensive as their AES-GCM-augmented variants, so these copies dominate the CPU bounce's fixed cost rather than RDMA or additional AES-GCM operations. The intra-GPU D2D copy also incurs a high latency similar to that of the D2H and H2D copies.

Figure 3.3 explains the latency overhead incurred under CC by separating command submission from on-device execution. For the confidential intra-GPU D2D copy at 32 B, median command submission takes 17.31 µs, while the median Copy Engine execution takes 2.21 µs. Command submission, therefore, accounts for 89% of the

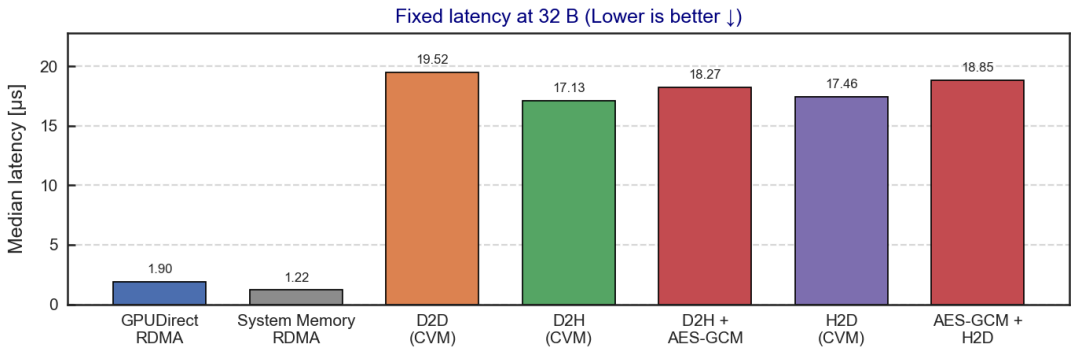


Figure 7.3: Measured fixed-latency parameters at 32 B.

fixed copy latency. The on-device component remains close to its non-confidential counterpart; almost all additional operations arise before or after device execution in the confidential command path described in Section 2.2.1.

Parameter	Measurement	Median latency
$\hat{\alpha}_{\mathrm{gdr}}$	GPUDirect RDMA	1.9 µs
$\hat{\alpha}_{\mathrm{D2H+enc}}$	D2H + AES-GCM	18.27 µs
$\hat{\alpha}_{\mathrm{rdma}}$	System Memory RDMA	1.22 µs
$\hat{\alpha}_{\mathrm{dec+H2D}}$	AES-GCM + H2D	18.85 µs
$\hat{\alpha}^{C}_{v_p \to v_p}$	D2D	19.52 µs

Table 7.2: Measured fixed-latency quantities used by the cost models.

Table 7.2 lists the measured latency quantities used to instantiate the cost models.

7.4 End-to-End Flow Completion Time

We first instantiate the cost models for both designs and the baseline datapaths, and then validate the paths that the testbed can measure. For GPUDirect RDMA, we compare the model's predictions with direct measurements. For the CPU bounce, we validate the one-sided model against a complete one-sided measurement. Because Design 1 cannot be measured directly, we validate its one-sided model against the closest implementable design. Finally, we compare the projected flow completion times of Designs 1 and 2 with those of the baselines.

7.4.1 Model Instantiation and Validation (Q1)

Instantiation. Substituting the parameters measured in Section 7.2 and Section 7.3 yields the GPUDirect cost model

$$T_{\mathrm{gdr}(n)} = 1.9\,\mu\mathrm{s} + \frac{n}{23.15\,\mathrm{GB/s}}, \tag{40}$$

the CPU bounce models

$$T^1_{\mathrm{bb}}(n) = 19.49\,\mu\mathrm{s} + \frac{n}{4.87\,\mathrm{GB/s}} \tag{41}$$

and

$$T^2_{\mathrm{bb}}(n) = 38.34\,\mu\mathrm{s} + \frac{n}{4.53\,\mathrm{GB/s}}, \tag{42}$$

as well as Design 1's one-sided cost model

$$T^1_{\mathrm{D1}}(n) = 21.42\,\mu\mathrm{s} + \frac{n}{23.15\,\mathrm{GB/s}}. \tag{43}$$

Under a homogeneous deployment in which both nodes have a GPU, the corresponding two-sided cost model is

$$T^2_{\mathrm{D1}}(n) = 40.94\,\mu\mathrm{s} + \frac{n}{23.15\,\mathrm{GB/s}}. \tag{44}$$

For Design 2, substituting the estimated projection costs and bandwidth ceiling yields:

$$\begin{aligned} \alpha_{\mathrm{D2}} &= \alpha_{\mathrm{gdr}} + \Delta\alpha^{\mathrm{IDE}} + \Delta\alpha^{\mathrm{ESP}} \\ &= 1.9\,\mu\mathrm{s} + 1.26\,\mu\mathrm{s} + 2\,\mu\mathrm{s} = 5.16\,\mu\mathrm{s} \\ \beta_{\mathrm{D2}} &= \min\!\left(\beta_{\mathrm{gdr}}, \beta^{\mathrm{IDE}}, \beta^{\mathrm{ESP}}\right) \\ &= \min(23.15\,\mathrm{GB/s}, 53.8\,\mathrm{GB/s}, 27.85\,\mathrm{GB/s}) = 23.15\,\mathrm{GB/s}. \end{aligned} \tag{45}$$

The end-to-end cost model is therefore

$$T^2_{\mathrm{D2}}(n) = 5.16\,\mu\mathrm{s} + \frac{n}{23.15\,\mathrm{GB/s}} \tag{46}$$

Validation. Figure 7.4 compares the GPUDirect cost model with the measurements used to parameterize it, measuring the approximation error introduced by fitting the measured curve with an (α, β) model. For the CPU bounce, we build the model from independently measured components and compare it with the complete one-sided path. This tests whether the independently measured components accurately predict the complete one-sided path. Design 1 cannot be measured directly because our setup

lacks the required unprotected video memory. We therefore validate its modeled one-sided variant, in which only one node has a GPU, against the closest measurable path: a sealed bounce from GPU memory through unprotected system memory, followed by an RDMA transfer to the other node's unprotected system memory.

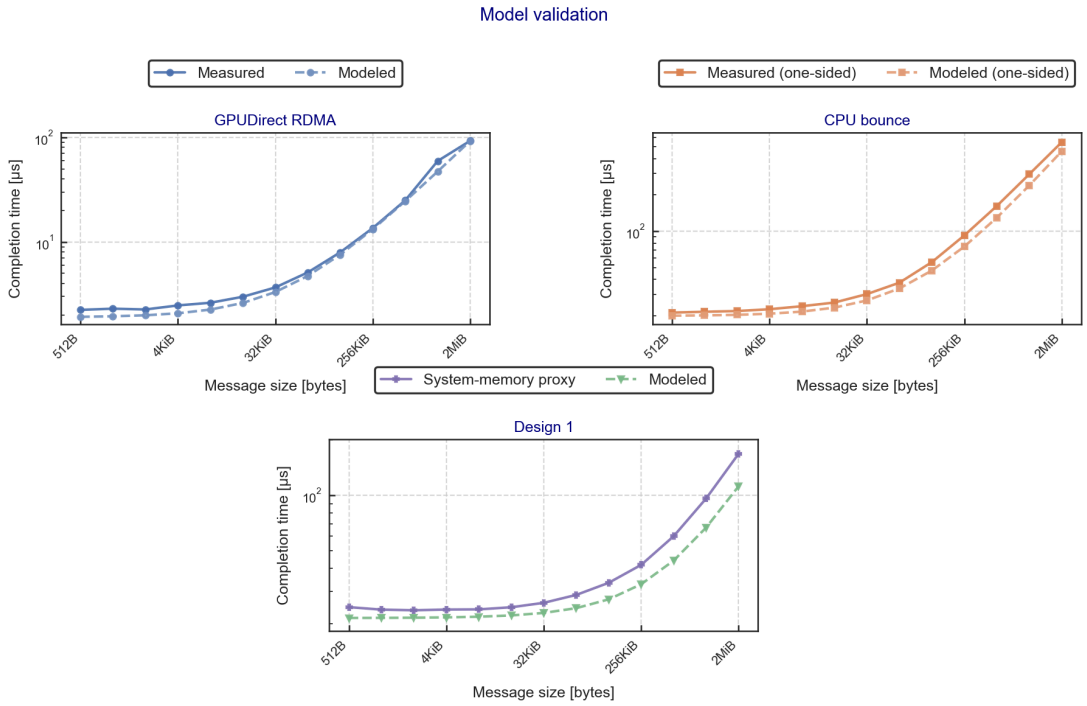


Figure 7.4: Model fit, validation and proxy comparison

The GPUDirect curve fit and one-sided CPU-bounce model predict their measured data paths with a geometric-mean residual of 12% and a geometric-mean error of 17%, respectively, supporting our modeling approach. The Design 1 projection differs from the non-equivalent system-memory proxy by 21%, with a −17% bias. This bias is expected: the proxy uses a slower D2H copy and system-memory RDMA, whereas Design 1 uses the faster intra-GPU D2D copy and GPUDirect. This comparison is therefore interpreted as a sanity check on the projection rather than a direct validation. Design 2 has no measurable proxy, so both designs remain projections.

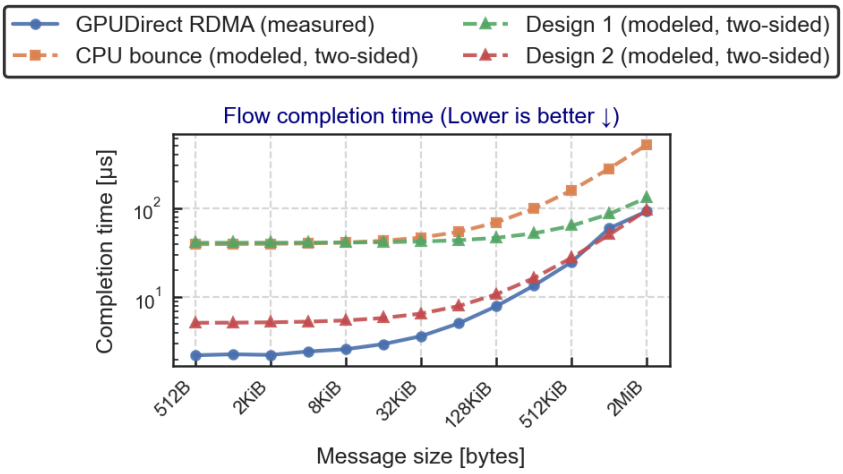


Figure 7.5: Flow completion time for GPUDirect, the CPU bounce, Design 1, and Design 2

7.4.2 Datapath Comparison (Q2)

Figure 7.5 compares measured GPUDirect with the modeled two-sided CPU bounce and the projected two-sided IronLink designs. For small messages, Design 1 incurs 2055% higher fixed latency than GPUDirect and 7% higher fixed latency than the CPU bounce. As detailed in Section 7.3, this overhead relative to GPUDirect stems from submitting a single confidential Copy Engine operation at each endpoint. Design 2 instead has 172% higher latency than GPUDirect, but 87% lower latency than the CPU bounce because its projected IDE and ESP processing adds much less fixed overhead than the confidential GPU copies.

For large messages, Design 1's ordering relative to the CPU bounce reverses while Design 2 remains faster. Both designs match the bandwidth of GPUDirect RDMA, which exceeds the CPU bounce bandwidth by 411%. At the largest evaluated message size of 2 MiB, Design 1 therefore achieves a projected flow completion time that is 74% lower than the modeled two-sided CPU bounce, while remaining 42% higher than measured GPUDirect. At the same size, Design 2 is 81% lower than the CPU bounce, 3% higher than GPUDirect, and 27% lower than Design 1. Because both designs and GPUDirect share the same bandwidth limit, the remaining gap is due to the fixed copy overhead, whose impact diminishes as message size increases.

7.5 Collective Communication (Q3)

7.5.1 Simulation Setup

We evaluate collective communication using the analytical backend of ASTRA-sim [84]. For each datapath, the simulated network link uses the (α, β) model instantiated from the measurements in Section 7.2 and Section 7.3. Each generated Chakra trace contains a single All-Reduce, All-Gather, Reduce-Scatter, or All-to-All operation. We evaluate GPU counts ranging from 8 to 256 and payload sizes from 1 KiB to 512 MiB.

Physical topology. ASTRA-sim applies the configured link latency for every physical hop [84]. However, our α represents the complete endpoint-to-endpoint datapath, including endpoint-specific latency such as processing and copies to or from a GPU. We therefore fix the physical topology to a fully connected fabric, so every logical transfer consists of exactly one hop.

Logical algorithms. On the fixed physical fabric, we evaluate ASTRA-sim's Ring and Halving-Doubling logical algorithms. Their concrete transfer schedules depend on the collective: Ring uses ring or pairwise-exchange schedules, whereas Halving-Doubling uses recursive halving and/or recursive doubling. ASTRA-sim does not support Halving-Doubling for All-to-All, so we omit this combination. We exclude ASTRA-sim's Direct implementation because it assumes that every rank can communicate with all other ranks simultaneously at full link bandwidth, which does not match our testbed, where all transfers from a rank share one NIC's bandwidth.

Table 7.3 lists the analytical communication time for each logical algorithm and collective. The expressions follow R. Thakur *et al.* [78] and P. Patarasuk and X. Yuan [67]. Here, p denotes the number of ranks, α the fixed per-message latency, $\overline{\beta}$ the transfer time per byte, and γ the per-byte local reduction cost. n denotes the collective data size: the input size for All-Reduce and Reduce-Scatter, the result size for All-Gather, and the per-rank data volume for All-to-All [78]. The recursive algorithms assume a power-of-two number of ranks, as used in our sweep [78].

Collective	Implementation	Communication cost
All-Gather	Ring	$(p-1)\alpha + \frac{p-1}{p}n\overline{\beta}$
All-Gather	Recursive doubling	$\lg p\alpha + \frac{p-1}{p}n\overline{\beta}$
Reduce-Scatter	Pairwise exchange	$(p-1)\alpha + \frac{p-1}{p}n\overline{\beta} + \frac{p-1}{p}n\gamma$
Reduce-Scatter	Recursive halving	$\lg p\alpha + \frac{p-1}{p}n\overline{\beta} + \frac{p-1}{p}n\gamma$
All-Reduce	Ring	$2(p-1)\alpha + 2\frac{p-1}{p}n\overline{\beta} + \frac{p-1}{p}n\gamma$
All-Reduce	Recursive halving/doubling	$2\lg p\alpha + 2\frac{p-1}{p}n\overline{\beta} + \frac{p-1}{p}n\gamma$
All-to-All	Pairwise exchange	$(p-1)\alpha + n\overline{\beta}$
All-to-All	Halving-Doubling	unsupported

Table 7.3: Analytical cost models of the evaluated collective algorithms.

7.5.2 Collective Completion Time

The following figures isolate the effects of payload size, datapath design, and GPU count on completion time.

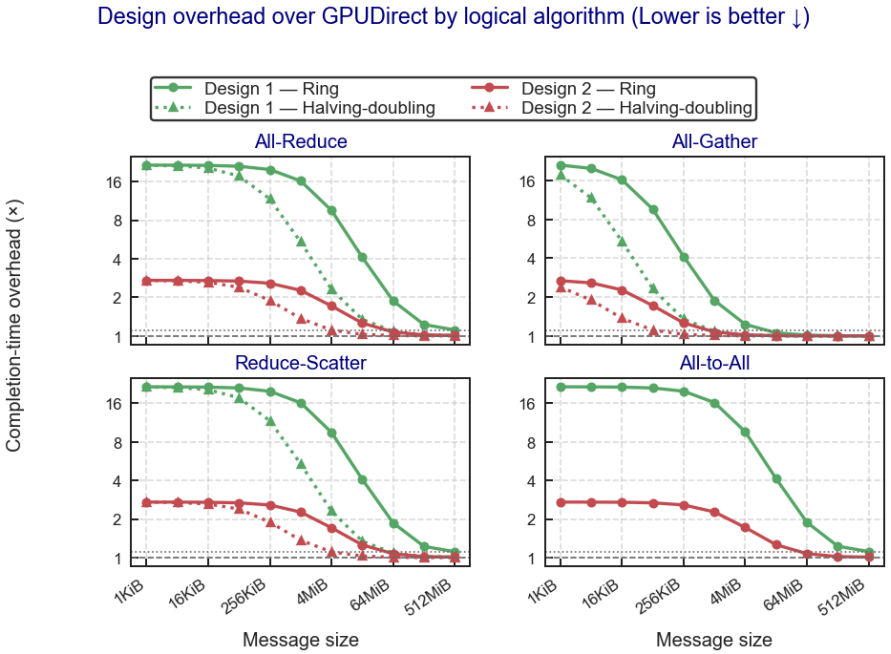


Figure 7.6: Projected Design 1 and Design 2 overhead over GPUDirect at 64 GPUs.

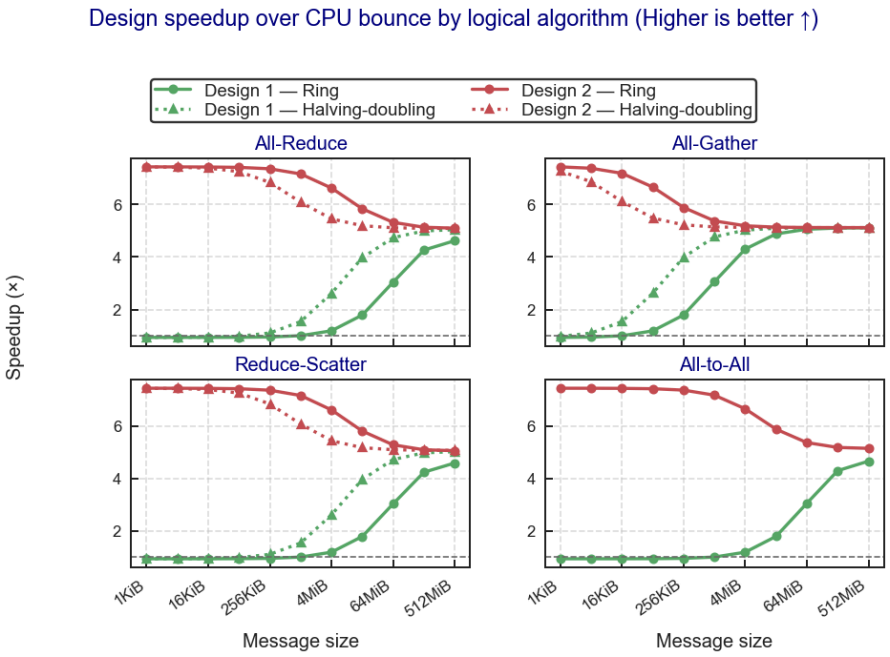


Figure 7.7: Projected Design 1 and Design 2 speedup over CPU bounce at 64 GPUs.

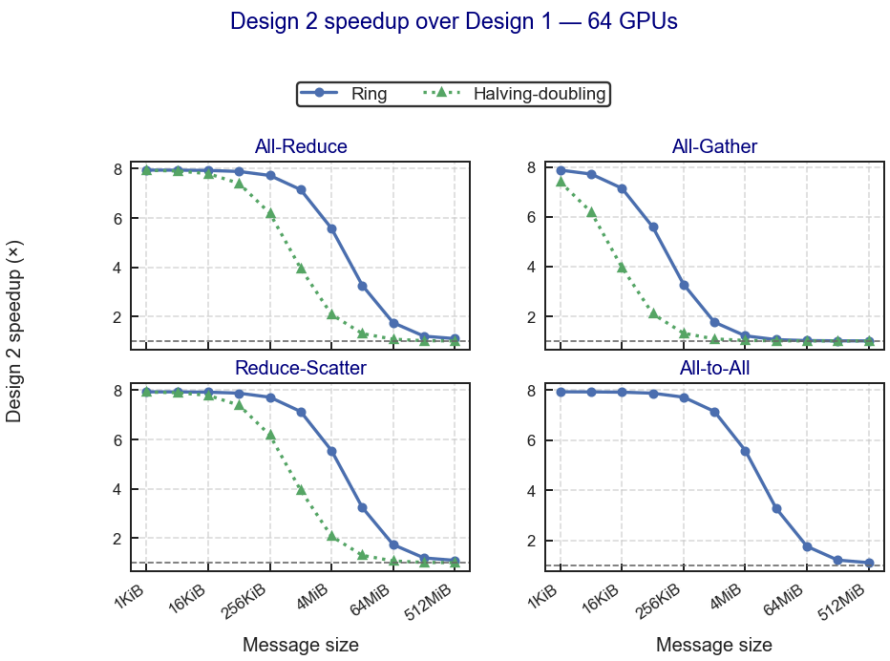


Figure 7.8: Projected Design 2 speedup over Design 1 at 64 GPUs.

Effect of payload size. At 1 KiB, Design 1 takes 17.7–21.5× as long as GPUDirect and 6% longer than the CPU bounce. Design 2 reduces this small-message overhead to 2.4–2.7× GPUDirect and is 7.3–7.4× faster than the CPU bounce. Equivalently, Design 2 is 7.4–7.9× faster than Design 1 at the smallest payload, as Figure 7.8 shows. As shown in Figure 7.6, the gap to GPUDirect narrows as payload size increases because Design 1's fixed copy cost is amortized, falling to an average 5% overhead at the largest evaluated payload. This behavior follows directly from the measurements in Section 7.2 and Section 7.3: Design 1 and GPUDirect have the same asymptotic bandwidth, whereas the CPU bounce remains limited by CUDA's confidential D2H path. Consequently, at 512 MiB, Design 1 has 78–80% lower completion time than the CPU bounce, equivalent to a 4.6–5.1× speedup, as shown in Figure 7.7. For a fixed rank count, Halving-Doubling approaches GPUDirect at smaller message sizes because it incurs the fixed per-message latency fewer times. The cost models in Table 7.3 explain this behavior: the latency term scales with $\lg(p)$ for Halving-Doubling, compared with $(p-1)$ for Ring, while both have the same asymptotic bandwidth term.

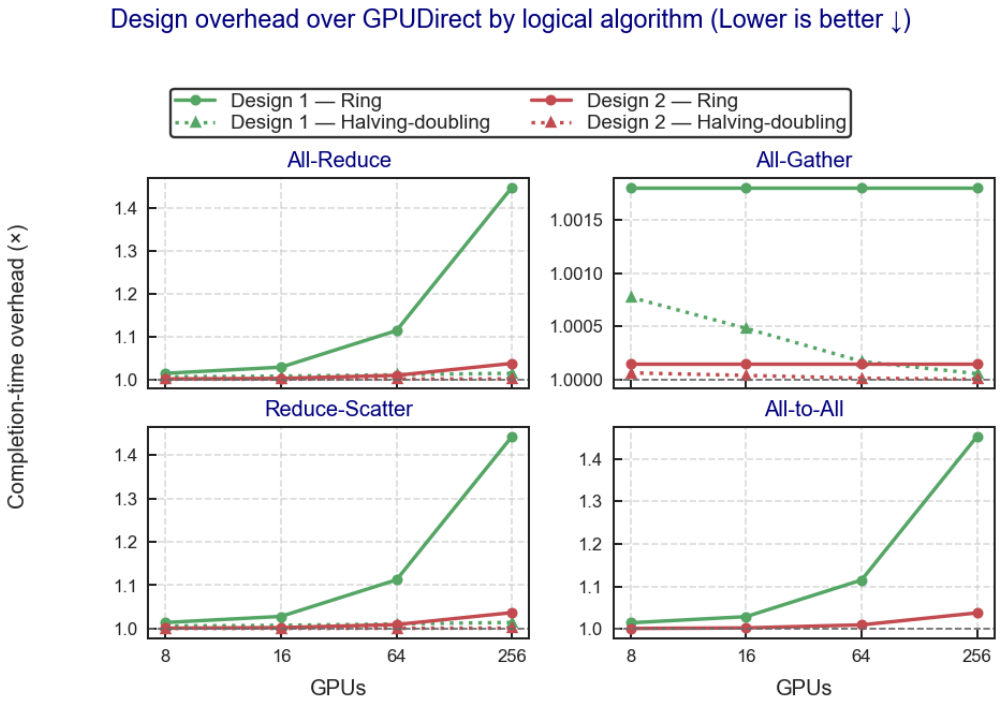


Figure 7.9: Projected Design 1 and Design 2 overhead over GPUDirect at 512 MiB.

Scaling with GPU count. Figure 7.9 fixes the payload at 512 MiB and shows the overhead of both designs over GPUDirect as the GPU count grows. For Ring All-Reduce, Reduce-Scatter, and All-to-All, the maximum overhead rises from 1% at 8 GPUs to 45% at 256 GPUs. Design 2 limits the maximum overhead at 256 GPUs to 4%. For a fixed payload, Ring charges each design's additional fixed latency $p-1$ times per phase for All-Reduce and Reduce-Scatter, so the overhead grows with p. Design 2 remains closer to GPUDirect because its per-transfer fixed cost is lower. Halving-Doubling instead charges the additional latency only $\lg p$ times per phase, keeping both designs close to GPUDirect. For All-Gather, each rank contributes m bytes, while n in Table 7.3 is the total result received by each rank and therefore $n = pm$. Substituting this value gives both Ring and Halving-Doubling the bandwidth term $(p-1)m\beta$. At 512 MiB, this term dominates: it grows at the same rate as Ring's latency term and faster than Halving-Doubling's $\lg p$ latency term, so increasing the GPU count does not increase the fixed-cost overhead.

Overall, Design 2 always matches or outperforms Design 1: its advantage is largest for small messages and Ring schedules because of its linear step count, which exposes fixed latency, and diminishes when bandwidth dominates.

7.6 Distributed ML Workloads (Q4)

7.6.1 Simulation Setup

Next, we evaluate how differences in the datapath across individual collectives affect the projected distributed training time. We use STAGE to generate Chakra traces from real model architectures and parallelization plans [52], then use ASTRA-sim to simulate their completion time [84]. The traces capture communication operations and dependencies but not computation, so the reported completion times are communication makespans. For each workload, we vary the logical collective algorithm and the datapath design. Everything else remains fixed. As in Section 7.5, we use the fully connected one-hop fabric and the parametrized (α, β) models of GPUDirect, CPU bounce, Design 1, and Design 2. As in Section 7.5, we consider Ring and Halving-Doubling as collective implementations and ignore Direct. Since Halving-Doubling does not support All-to-All, W5 is evaluated only with Ring.

We evaluate the five workloads in Table 7.4. They cover pipeline parallelism, Fully Sharded Data Parallel (FSDP), tensor and sequence parallelism, hybrid parallelism, and expert parallelism. STAGE reports per-GPU collective errors between 0% and 2% for these exact configurations [52].

ID	Model	Parallelization	STAGE comm. error
W1	GPT-3 5B	PP=8	0.000%
W2	GPT-3 5B	FSDP=8	0.346%
W3	GPT-3 175B	TP=32 + SP	0.055%
W4	GPT-3 175B	DP=2, TP=4, PP=8 + SP	0.043%
W5	Mixtral 8x7B	EP=8, PP=4	1.399%

Table 7.4: Selected end-to-end workloads.

The five workloads use different architectures: Pipeline Parallelism (PP) partitions model layers into pipeline stages and divides training batches into microbatches that are pipelined through those stages [52, 55]. FSDP shards model parameters and gradients across workers, gathers parameters as needed for computation, and reduce-scatters gradients after backpropagation [91]. Tensor Parallelism (TP) partitions the model weights across devices and replicates input data. Activations are exchanged after every layer using All-Reduce [52, 55]. Sequence Parallelism (SP) partitions the input sequence so each device processes a different subset of tokens and replaces All-Reduce with All-Gather and Reduce-Scatter [44]. Expert Parallelism (EP) distributes the experts of a Mixture of Experts (MoE) model across devices. A router assigns tokens to experts, and All-to-All communication is used to send the input and collect the computed output [52].

7.6.2 Results

Figures 7.10 and 7.11 compare the completion times of both designs against GPUDirect and CPU bounce, respectively, across workloads and logical algorithms.

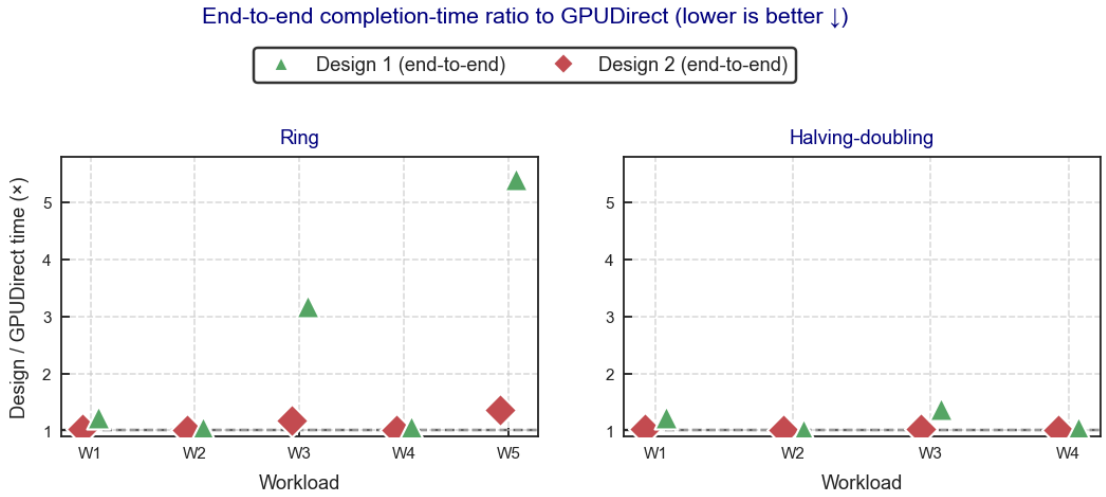


Figure 7.10: Projected Design 1 and Design 2 completion-time ratio relative to GPUDirect.

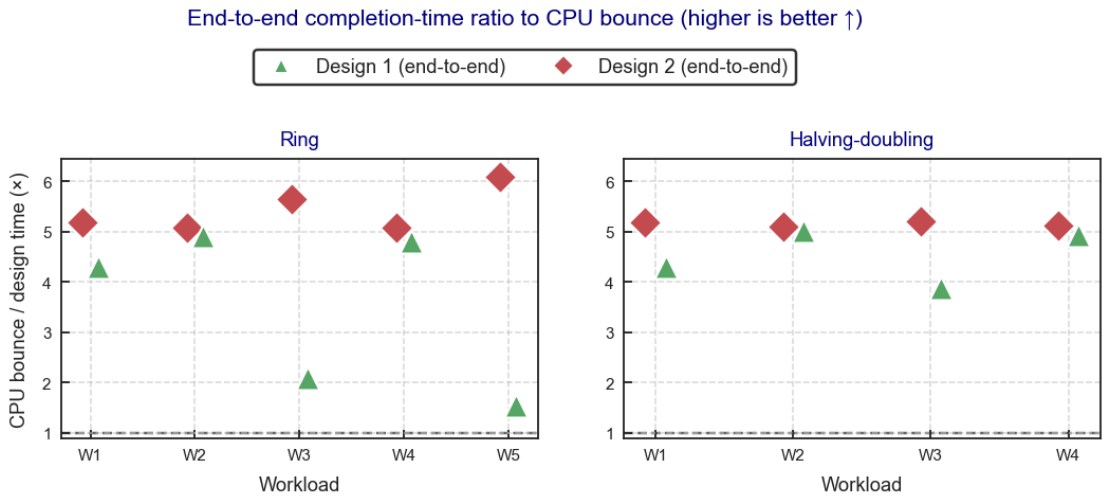


Figure 7.11: Projected CPU-bounce-to-design completion-time ratio.

Across the 9 selected workload/algorithm combinations, Design 1 has a median overhead of 23% over GPUDirect and a median speedup of 4.3× over CPU bounce, as shown

in Figures 7.10 and 7.11. The outcomes span from only 2% overhead over GPUDirect to 441%, while the maximum speedup over CPU bounce is 5×.

Design 2 reduces the median overhead over GPUDirect to 2%, with a range of <1%–37%, and raises the median CPU bounce speedup to 5.2×, with a range of 5.1–6.1×. Across all selected points, it is 1–4× faster than Design 1, with a median of 1.2×.

Workloads near the 1x line in Figure 7.10 are bandwidth-sensitive: the designs approach GPUDirect while substantially outperforming the lower-bandwidth CPU bounce in Figure 7.11. Higher overheads indicate fixed-latency-sensitive communication patterns that invoke enough small transfers for the additional per-transfer latency to remain visible.

Under Design 1, W2 and W4 occupy the bandwidth-sensitive region independent of the collective implementation. W3 moves from the fixed-latency-sensitive to the bandwidth-sensitive region when Ring is replaced with Halving-Doubling, and W5 is the most extreme case in the fixed-latency-sensitive region. Across all workloads, Design 2 is closer to GPUDirect and achieves greater speedups over the CPU bounce than Design 1.

7.6.3 Interpretation

As established in Section 7.5, Halving-Doubling reduces the number of All-Gather, Reduce-Scatter, and All-Reduce steps from linear to logarithmic while retaining the same asymptotic bandwidth term. W1 is implementation-invariant because PP relies on point-to-point transfers for activations and gradients [55], which connect fixed sender–receiver pairs and eliminate the need for a collective schedule. W2 uses All-Gather operations for parameters and Reduce-Scatter operations for gradients [91]. Since it is bandwidth-sensitive, the latency terms can be omitted, leaving only the matching bandwidth terms (Table 7.3), resulting in very little difference between Ring and Halving-Doubling. W3 invokes collectives in every transformer layer across a 32-rank TP+SP group [55]. Consequently, reducing the step count from linear to logarithmic removes many fixed-latency charges and explains the 2.5× speedup. W4 instead uses four-rank TP groups and adds PP and Data Parallelism (DP), so the collective step-count advantage is smaller, and part of its communication is point-to-point. W5 routes token activations through expert-parallel using All-to-All communication rather than moving complete parameters or gradients, which is consistent with its fixed-latency-sensitive result [39, 52].

8 Discussion

8.1 Limitations

GPU firmware support for Design 1. Design 1 requires capabilities that the current NVIDIA GPU firmware does not provide. The existing stack derives Copy Engine key bundles from the CVM–GPU SPDM session and installs a fixed set of keys for each LCE [28, 31]. In contrast, Design 1 requires the GPU to establish and maintain several SPDM sessions concurrently and to bind a channel, or an LCE used by that channel, to the key bundle derived from a selected session. At the time of writing, the firmware and driver interface exposes neither GPU-to-GPU session establishment nor the installation, selection, and lifecycle management of multiple peer-specific LCE key bundles [28, 31]. Design 1, therefore, requires extensions to the GSP firmware and GPU driver and cannot be implemented unchanged on the evaluated platform.

Blackwell Trusted I/O does not support the required GPU to NIC path. NVIDIA describes TDISP and IDE-based protection between a CVM and a Blackwell GPU to eliminate the need for a bounce buffer and slow software cryptography [58]. However, the current NVIDIA General Availability (GA) release still describes communication between the GPU and CVM in Blackwell CC mode via bounce buffers [16]. Additionally, we are not aware of any public documentation that describes a protected NIC-to-GPU P2P path or a TDISP-based interface through which a NIC can access the CPR. Design 2 consequently builds on TDISP and IDE specifications, although the corresponding functionality is not yet available or implemented in NVIDIA firmware [62, 63].

Availability of NICs with Trusted I/O support. At the time of writing, we are not aware of any commercially available RDMA NICs that document all capabilities required by Design 2: TDISP device admission, support for P2P IDE binding, and full offload for IPsec over RoCEv2. Prior work similarly reports the lack of a released TDISP-capable setup for experimental evaluation [7, 50].

Model based evaluations. Because of the testbed and firmware limitations described above, we cannot deploy and measure a complete two-sided datapath. Instead, we measure its individual stages using microbenchmarks, compose them into cost models, and use these models in simulation. This assumes that the measured parameters are

representative of the complete datapath, that the homogeneous deployment assumption holds, and that resource contention does not reduce pipelining efficiency. On the evaluated 200 GbE link, both designs share the same network-limited asymptotic bandwidth; therefore, Design 2 matches or outperforms Design 1 due to its lower fixed latency. Faster links could expose a difference in asymptotic bandwidth and change this ordering.

Simulation constraints. The collective and distributed workload results are obtained from generated Chakra traces and ASTRA-sim's analytical backend rather than real-world executions [52, 84]. The simulations use a fully connected one-hop topology and a limited set of collective algorithms, omitting routing, congestion, and packet loss. They provide controlled comparisons between the evaluated data paths, but do not predict the exact completion time for a production deployment.

8.2 Future Directions

Realize Designs 1 and 2. The first direction for future work is a complete implementation of Design 1. This requires extending the GPU firmware and driver interfaces to support multiple SPDM sessions and to select the corresponding key bundle for each LCE and channel. Design 2 should be implemented once GPUs, NICs, and platforms expose the required Trusted I/O capabilities.

Prototype-based evaluation. Once the two designs can be deployed, their performance should be evaluated using complete two-sided transfers. This evaluation should measure flow completion times, integrate the datapaths with NVIDIA Collective Communications Library (NCCL), and execute representative distributed training workloads. The resulting measurements would enable validation of the analytical cost models used in this work.

Broader simulation. By separating hop and endpoint latency in the cost models, the simulation could be extended beyond a fully connected one-hop topology, enabling evaluation across different network architectures. Future experiments should consider realistic multi-hop topologies, routing, congestion, and additional collective algorithms.

Collective-aware optimization. The results show that the overhead of a secure datapath depends on the collective algorithm used. Future work could therefore co-design the communication layer and datapath by adapting the collective algorithm, chunk size, batching, and scheduling based on the characteristics of the transfer. Such an approach could select different configurations depending on the message size, collective,

and workload. One path toward this would be to incorporate the cost models of Design 1 and 2 into NCCL's existing cost model, allowing NCCL to account for transfer costs when selecting algorithms. SCALE demonstrates a similar extension using encryption-specific parameters [60].

Session Scalability. Finally, the proposed pairwise sessions could pose challenges at scale with many GPUs, especially in a fully connected setting. Future work should investigate the cost of establishing and maintaining sessions between a large number of GPUs, as well as mechanisms for key rotation, re-attestation, and failure recovery.

9 Related Work

Confidential GPUs. Prior work protects accelerator execution through isolated GPU contexts, controlled host–device interfaces, bound or composed CPU/device enclaves, protected memory or validated execution, accelerator roots of trust, and intermediate security controllers [6, 20, 36, 37, 38, 45, 51, 71, 74, 79, 80, 82, 85, 89, 93]. Other work prevents sensitive information from leaking through GPU communication and memory-access patterns in remotely executed programs and video analytics [33, 65]. NVIDIA Hopper makes this model commercially available through protected GPU memory, device attestation, authenticated command submission, and encrypted CPU-to-GPU transfers [21, 28]. These works establish local boundaries, but do not protect GPU endpoints at both ends of inter-node GPUDirect RDMA.

Composite Enclaves protects applications spanning a CPU and specialized hardware by composing remotely attestable unit enclaves and communicating through access-controlled shared-memory regions. The selected device firmware and driver become explicit parts of a configurable TCB [71]. In comparison, IronLink assumes an existing CVM and confidential GPU boundary rather than defining a new GPU TEE, and focuses on communication between two remote GPU endpoints. Design 1 terminates the data session in the GPUs while leaving the NIC outside the TCB, whereas Design 2 extends the TCB to include the admitted NIC.

Confidential GPU communication. Work on confidential GPU communication overlaps encryption with transfer and computation, removes redundant CPU-TEE cryptography, characterizes transfer and runtime overheads, or adapts LLM serving to these costs [53, 59, 75, 81, 88, 90, 94]. Multi-GPU work reduces security-metadata traffic, characterizes secure collectives, or co-designs encryption and collective algorithms [47, 54, 60]. These evaluations use simulation or emulation rather than deployed multi-GPU confidential computing.

Fastrack establishes a cryptographic channel directly between the remote user and GPU TEE, eliminating CPU-side decryption and re-encryption. Its physical data path nevertheless stages the ciphertext in a host-memory bounce buffer before PCIe DMA to the GPU [81]. In comparison, IronLink Design 1 also terminates the secure channel in the GPU and avoids CPU-side payload cryptography, but keeps both Exposure Buffers in GPU memory and uses GPUDirect RDMA to transfer records between hosts.

SCALE protects distributed ML communication by combining the GPU's hardware encryption path with software AES-GCM and GHASH on otherwise idle GPU resources, precomputing One Time Pads (OTPs) in protected GPU memory and selecting collective algorithms using encryption-aware cost functions [60]. Like IronLink Design 1, SCALE keeps keys and cryptographic processing at the GPU endpoint, but it uses CUDA SMs alongside hardware engines and co-designs complete collective operations, whereas IronLink only provides a point-to-point secure channel and uses the hardware cryptography of the Copy Engines in Design 1. SCALE evaluates encrypted multi-GPU execution through trace-driven simulation calibrated and generated with H100/H200 NCCL measurements because the evaluated software stack does not support multi-GPU execution, whereas IronLink isolates and models the underlying inter-node GPUDirect RDMA channel on which higher-level collectives could be built.

Secure RDMA and trusted NICs. Prior work exposes the lack of protection in RDMA, offloads authenticated and optional encryption to trusted NICs or applies topology based validation and policy enforcement without confidentiality [70, 76, 86]. Programmable-NIC systems provide tenant isolation, transferable authentication, secure function offload and leakage-resistant I/O sandboxing [27, 48, 49, 92]. Adjacent work uses GPUDirect for Multi Party Computation or explores inline RDMA payload encryption and NIC side preprocessing followed by direct DMA into GPU memory [30, 68].

Among these systems, sRDMA enables RDMA source authenticity, payload integrity, freshness and confidentiality by performing symmetric cryptography in trusted RNICs [76]. In comparison, IronLink Design 2 also terminates wire protection at a trusted NIC, but does not treat the device path as trusted: it admits the GPU and NIC, restricts the NIC's access to the GPU Exposure Buffer, protects peer-to-peer PCIe traffic with IDE, and protects RoCEv2 between NICs using ESP.

Trusted I/O. Trusted I/O extends attestation across CPU/device components, protects external GPUs, or interposes a security controller between the host and its devices [6, 22, 71]. ACAI attaches an accelerator to protected host memory. IronLink Design 2 instead admits a NIC with restricted access to a protected GPU buffer and extends protection across the network, while Design 1 exposes only GPU-encrypted records to an untrusted NIC. TDX Connect and SEV-TIO combine SPDM, TDISP, and PCIe IDE to attach devices to CVMs [3, 13] and NVIDIA describes TDISP and IDE support for Blackwell GPUs [58]. Other systems optimize I/O through untrusted devices or offload secure storage to a trusted SmartNIC [7, 50]. No evaluated system currently demonstrates the TDISP-enabled SmartNIC datapath required by IronLink.

10 Conclusion

We presented **IronLink**, an architecture for secure GPU-to-GPU RDMA between CVMs. It keeps the CPU on the control path and provides two alternatives: In-GPU Bounce seals records in GPU memory while keeping the NIC untrusted, whereas Trusted NIC admits an attested NIC and combines direct access to GPU memory with PCIe IDE and NIC offloaded ESP. We identify software cryptography as the CPU bounce's bandwidth bottleneck and confidential command submission as the dominant fixed cost for small transfers under CC. Across the selected distributed ML traces, In-GPU Bounce and Trusted NIC achieve a projected median speedup of 4.3× and 5.2× over the CPU bounce, with median overheads of 23% and 2% over GPUDirect. Since current hardware lacks the required firmware and Trusted-I/O support, the complete datapaths remain projections. IronLink provides a concrete design space and implementation roadmap for confidential GPU networking.

A NVIDIA Driver Source Excerpts

```
//
// Retrieve IV from keystore as it needs to be transmitted to GPU.
// On Hopper this will also retrieve the key and IV mask.
// On Blackwell and later the key and IV mask are retrieved
// after a successful control call to GSP-Resource Manager (RM) to ensure that
// the keystore and keymanager keyseeds stay in sync.
//
NV_ASSERT_OK_OR_GOTO(
    status,
    confComputeKeyStoreRetrieveViaChannel_HAL(
      pConfCompute, pKernelChannel, ROTATE_IV_ALL_VALID,
      CHANNEL_IV_OPERATION_INCLUDE_ONLY, &pKernelChannel->clientKmb
    );
    cleanup
);

portMemCopy(pChannelGpfifoParams->encryptIv,
          sizeof(pChannelGpfifoParams->encryptIv),
          pKernelChannel->clientKmb.encryptBundle.iv,
          sizeof(pKernelChannel->clientKmb.encryptBundle.iv));

portMemCopy(pChannelGpfifoParams->decryptIv,
          sizeof(pChannelGpfifoParams->decryptIv),
          pKernelChannel->clientKmb.decryptBundle.iv,
          sizeof(pKernelChannel->clientKmb.decryptBundle.iv));
```

Listing A.1: CPU RM keystore IV state is copied into the GPU channel-allocation parameters.
`kernel_channel.c:782-803`

```
channelCounter++;

switch ((*pKeyStore)[slotNumber].type)
{
    ...
    case CRYPT_COUNTER:
        (*pKeyStore)[slotNumber].cryptBundle.iv[2] = NvU64_HI32(channelCounter);
        (*pKeyStore)[slotNumber].cryptBundle.iv[1] = NvU64_LO32(channelCounter);
        break;
    ...
}
```

Listing A.2: The GPU driver assigns the 64-bit channel counter to the raw IV state. `conf_compute_keystore_gh100.c:697-708`

```
confComputeKeyStoreDepositIvMask_HAL(pConfCompute,
    CC_GKEYID_GEN(ccKeyspaceLCEIndex, CC_LKEYID_LCE_H2D_USER),
    (void*)&params.ivMaskSet[0].ivMask);

confComputeKeyStoreDepositIvMask_HAL(pConfCompute,
    CC_GKEYID_GEN(ccKeyspaceLCEIndex, CC_LKEYID_LCE_D2H_USER),
    (void*)&params.ivMaskSet[1].ivMask);
```

Listing A.3: GSP-derived IV masks are saved in the GPU driver. `conf_compute_gh100.c:138-152`

Abbreviations

AEAD Authenticated Encryption with Associated Data
ATS Address Translation Service
CC Confidential Computing
CCA Confidential Compute Architecture
CE Copy Engine
CGPU Confidential GPU
CplD Completion with Data
CPR Compute-Protected Region
CQ Completion Queue
CQE Completion Queue Entry
CVM Confidential Virtual Machine
D2D Device-to-Device
D2H Device-to-Host
DG Design Goal
DOE Data Object Exchange
DP Data Parallelism
DSM Device Security Module
DTM Device Trust Manager
ECRC End-to-End Cyclic Redundancy Check
EP Expert Parallelism
ESP Encapsulating Security Payload
FLIT Flow Control Unit
FSDP Fully Sharded Data Parallel
GA General Availability
GCM Galois/Counter Mode
GDR GPUDirect RDMA
GPFIFO General-Purpose FIFO
GSP GPU System Processor
H2D Host-to-Device
HBM High Bandwidth Memory
IDE Integrity and Data Encryption

IKEv2 Internet Key Exchange version 2
ILC IronLink Channel
IOMMU Input/Output Memory Management Unit
IPsec Internet Protocol Security
IV Initialization Vector
LCE Logical Copy Engine
LCIC Launch Confirmation Indicator Channel
LCRC Link Cyclic Redundancy Check
MAC Message Authentication Code
MMIO Memory-Mapped I/O
MoE Mixture of Experts
MR Memory Region
MRd Memory Read
MWr Memory Write
NCCL NVIDIA Collective Communications Library
NIC Network Interface Card
NVLink NVIDIA Link
OTP One Time Pad
P2P peer-to-peer
PCRC Protected Cyclic Redundancy Check
PD Protection Domain
PEF Protected Execution Facility
PP Pipeline Parallelism
QP Queue Pair
RA-TLS Remote Attestation TLS
RC Reliable Connection
RDMA Remote Direct Memory Access
RID Requester ID
rkey remote key
RM Resource Manager
RMAPI GPU Resource Manager API
RoCE RDMA over Converged Ethernet
SEC2 Security Processor 2
SEV Secure Encrypted Virtualization
SM Streaming Multiprocessor
SN Sequence Number

SNP Secure Nested Paging
SP Sequence Parallelism
SPDM Security Protocol and Data Model
TCB Trusted Computing Base
TDI TEE Device Interface
TDISP TEE Device Interface Security Protocol
TDX Trust Domain Extensions
TEE Trusted Execution Environment
TLP Transaction Layer Packet
TLS Transport Layer Security
TP Tensor Parallelism
TSM TEE Security Module
UDP User Datagram Protocol
VMM Virtual Machine Monitor
WLC Work Launch Channel

List of Figures

2.1 TDISP architecture 8
2.2 TDISP workflow 9
3.1 GPUDirect RDMA datapath. 12
3.2 Secure CPU-bounce fallback under GPU CC. 13
3.3 Median command-submission and Copy Engine latency of a 32 B copy. 15
4.1 IronLink architecture overview 18
5.1 IronLink Design 1 architecture overview 21
5.2 Sender-side channel-table entry c 24
6.1 IronLink Design 2 architecture overview 28
6.2 Steps 1–3 30
6.3 Steps 4–6 32
7.1 Measured write bandwidth for GPUDirect RDMA, System Memory RDMA, and the one-sided CPU bounce. 41
7.2 Measured D2H bandwidth for D2H (VM), Sealed CE copy D2H (CVM), D2H (CVM), and D2H + AES-GCM. 41
7.3 Measured fixed-latency parameters at 32 B. 43
7.4 Model fit, validation and proxy comparison 45
7.5 Flow completion time for GPUDirect, the CPU bounce, Design 1, and Design 2 46
7.6 Projected Design 1 and Design 2 overhead over GPUDirect at 64 GPUs. 48
7.7 Projected Design 1 and Design 2 speedup over CPU bounce at 64 GPUs. 49
7.8 Projected Design 2 speedup over Design 1 at 64 GPUs. 49
7.9 Projected Design 1 and Design 2 overhead over GPUDirect at 512 MiB. 50
7.10 Projected Design 1 and Design 2 completion-time ratio relative to GPUDirect. 53
7.11 Projected CPU-bounce-to-design completion-time ratio. 53
A.1 CPU RM keystore IV state is copied into the GPU channel-allocation parameters. `kernel_channel.c:782-803` 61
A.2 The GPU driver assigns the 64-bit channel counter to the raw IV state. `conf_compute_keystore_gh100.c:697-708` 62
A.3 GSP-derived IV masks are saved in the GPU driver. `conf_compute_gh100.c:138-152` 62

List of Tables

2.1 Notation 11
7.1 Measured bandwidth quantities used to parametrize the cost models 42
7.2 Measured fixed-latency quantities used by the cost models. 43
7.3 Analytical cost models of the evaluated collective algorithms. 48
7.4 Selected end-to-end workloads. 52

Bibliography

[1] *IPsec Full Offload | MLNX_OFED Software*. (Jan. 2024). url: https://networking-docs.nvidia.com/mlnxofedswum/24.10-5.1.6.1lts/ipsec-packet-offload.

[2] A. Alexandrov, M. F. Ionescu, K. E. Schauser, and C. Scheiman. "LogGP: incorporating long messages into the LogP model—one step closer towards a realistic model for parallel computation." In: *Proceedings of the seventh annual ACM symposium on Parallel algorithms and architectures*. (July 1995). pp. 95–105. doi: 10.1145/215399.215427. url: https://dl.acm.org/doi/10.1145/215399.215427.

[3] AMD. *AMD SEV-TIO: Trusted I/O for Secure Encrypted Virtualization*. url: https://www.amd.com/content/dam/amd/en/documents/developer/sev-tio-whitepaper.pdf.

[4] AMD. "AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More." (2020). url: https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.

[5] E. Agostini, D. Rossetti, and S. Potluri. "Offloading Communication Control Logic in GPU Accelerated Applications." In: *2017 17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. (May 2017). pp. 248–257. doi: 10.1109/CCGRID.2017.29. url: https://ieeexplore.ieee.org/document/7973709.

[6] R. Bahmani, F. Brasser, G. Dessouky, P. Jauernig, M. Klimmek, A.-R. Sadeghi, and E. Stapf. "CURE: A Security Architecture with CUstomizable and Resilient Enclaves." (2021). pp. 1073–1090. url: https://www.usenix.org/conference/usenixsecurity21/presentation/bahmani.

[7] M. Chrapek, M. Orenbach, A. Atamli, M. Copik, M. Khalilov, F. Alder, and T. Hoefler. *Hazel: Secure and Efficient Disaggregated Storage*. (Feb. 2026). doi: 10.48550/arXiv.2510.18756. url: http://arxiv.org/abs/2510.18756.

[8] linux-rdma contributors. *linux-rdma/perftest*. (June 2026). url: https://github.com/linux-rdma/perftest.

[9] I. Corporation. *Intel® TDX Connect Architecture Specification*. (Mar. 2023). url: https://www.intel.com/content/www/us/en/content-details/773614/intel-tdx-connect-architecture-specification.html.

[10] N. Corporation. *RDMA over Converged Ethernet (RoCE)*. (Feb. 2024). url: https://docs.nvidia.com/networking/display/MLNXOFEDv23070512/RDMA+over+Converged+Ethernet+(RoCE).

[11] N. Corporation. *1. Overview — GPUDirect RDMA 13.3 documentation*. (June 2026). url: https://docs.nvidia.com/cuda/gpudirect-rdma/.

[12] I. Corporation. *Intel® Core™ Ultra 200S, Intel® Core™ Ultra 200S Plus, Intel® Core™ Ultra 200HX and 200HX Plus Series Processors Datasheet, Volume 1 of 2*. url: https://www.intel.com/content/www/us/en/content-details/832586/intel-core-ultra-200s-intel-core-ultra-200s-plus-intel-core-ultra-200hx-and-200hx-plus-series-processors-datasheet-volume-1-of-2.html.

[13] I. Corporation. *Intel® TDX Connect TEE-IO Device Guide*. url: https://www.intel.com/content/www/us/en/content-details/861654/intel-tdx-connect-tee-io-device-guide.html.

[14] I. Corporation. *Intel® Trust Domain Extensions (Intel® TDX)*. url: https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html.

[15] N. Corporation. *NVIDIA H100 GPU*. url: https://www.nvidia.com/en-us/data-center/h100/.

[16] N. Corporation. *NVIDIA Trusted Computing Solutions R595 GA Release Notes*. (Apr. 2026). url: https://docs.nvidia.com/595trd1-trusted-computing-solutions-release-notes.pdf.

[17] N. Corporation. *NVIDIA/nvbandwidth*. (June 2026). url: https://github.com/NVIDIA/nvbandwidth.

[18] N. Corporation. *Supported Interfaces | NVIDIA BlueField-3 Networking Platform User Guide*. (July 2026). url: https://networking-docs.nvidia.com/bf3dpu/supported-interfaces.

[19] D. Culler, R. Karp, D. Patterson, A. Sahay, K. E. Schauser, E. Santos, R. Subramonian, and T. von Eicken. "LogP: towards a realistic model of parallel computation." In: *Proceedings of the fourth ACM SIGPLAN symposium on Principles and practice of parallel programming*. (July 1993). pp. 1–12. doi: 10.1145/155332.155333. url: https://dl.acm.org/doi/10.1145/155332.155333.

[20] Y. Deng, C. Wang, S. Yu, S. Liu, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang. "StrongBox: A GPU TEE on Arm Endpoints." In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. (Nov. 2022). pp. 769–783. doi: 10.1145/3548606.3560627. url: https://dl.acm.org/doi/10.1145/3548606.3560627.

[21] G. Dhanuskodi, S. Guha, V. Krishnan, A. Manjunatha, R. Nertney, M. O'Connor, and P. Rogers. "Creating the First Confidential GPUs." In: *Commun. ACM*, 67 (1). (Dec. 2023). pp. 60–67. issn: 0001-0782. doi: 10.1145/3626827. url: https://doi.org/10.1145/3626827.

[22] A. Dhar, S. Sridhara, S. Shinde, S. Capkun, and R. Andri. "Confidential Computing with Heterogeneous Devices at Cloud-Scale." In: *2024 Annual Computer Security Applications Conference (ACSAC)*. (Dec. 2024). pp. 102–116. issn: 2576-9103. doi: 10.1109/ACSAC63791.2024.00025. url: https://ieeexplore.ieee.org/abstract/document/10917655.

[23] DMTF. *Secured Messages using SPDM Specification 1.2.0*. (June 2024). url: https://www.dmtf.org/sites/default/files/standards/documents/DSP0277_1.2.0.pdf.

[24] DMTF. *Security Protocol and Data Model (SPDM) Specification 1.4.1*. (June 2026). url: https://www.dmtf.org/sites/default/files/standards/documents/DSP0274_1.4.1.pdf.

[25] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. (Nov. 2007). doi: 10.6028/NIST.SP.800-38D. url: https://csrc.nist.gov/pubs/sp/800/38/d/final.

[26] S. Ghosh, L. S. Kida, S. J. Desai, and R. Lal. *A >100 Gbps Inline AES-GCM Hardware Engine and Protected DMA Transfers between SGX Enclave and FPGA Accelerator Device*. (2020). url: https://eprint.iacr.org/2020/178.

[27] D. Giantsidi, J. Pritzi, F. Gust, A. Katsarakis, A. Koshiba, and P. Bhatotia. *TNIC: A Trusted NIC Architecture*. (Feb. 2025). doi: 10.48550/arXiv.2502.05338. url: http://arxiv.org/abs/2502.05338.

[28] Z. Gu, E. Valdez, S. Ahmed, J. J. Stephen, M. Le, H. Jamjoom, S. Zhao, and Z. Lin. *Blueprint, Bootstrap, and Bridge: A Security Look at NVIDIA GPU Confidential Computing*. (Apr. 2026). doi: 10.48550/arXiv.2507.02770. url: http://arxiv.org/abs/2507.02770.

[29] C. Guo, H. Wu, Z. Deng, G. Soni, J. Ye, J. Padhye, and M. Lipshteyn. "RDMA over Commodity Ethernet at Scale." In: *Proceedings of the 2016 ACM SIGCOMM Conference*. (Aug. 2016). pp. 202–215. doi: 10.1145/2934872.2934908. url: https://dl.acm.org/doi/10.1145/2934872.2934908.

[30] M. J. Heer, B. Ramhorst, Y. Zhu, L. Liu, Z. Hu, J. Dann, and G. Alonso. "RoCE BALBOA: Service-Enhanced RDMA Offload Engine for Data Center SmartNICs." (2026). pp. 1495–1512. url: https://www.usenix.org/conference/osdi26/presentation/heer.

[31] A. Hendrickson, V. Kulkarni, G. Dhanuskodi, N. Cherukuri, W. Gandhi, and R. Wong. *Fairly utilizing multiple contexts sharing cryptographic hardware*. (Apr. 2024). url: https://patents.google.com/patent/US11966480B2/en.

[32] R. W. Hockney. "The communication challenge for MPP: Intel Paragon and Meiko CS-2." In: *Parallel Computing*, 20 (3). (Mar. 1994). pp. 389–398. issn: 0167-8191.

doi: 10.1016/S0167-8191(06)80021-9. url: https://www.sciencedirect.com/science/article/pii/S0167819106800219.

[33] T. Hunt, Z. Jia, V. Miller, A. Szekely, Y. Hu, C. J. Rossbach, and E. Witchel. "Telekine: Secure Computing with Cloud GPUs." (2020). pp. 817–833. url: https://www.usenix.org/conference/nsdi20/presentation/hunt.

[34] D. Iliadis-Apostolidis, D. C. Lawo, S. Kosta, I. Tafur Monroy, and J. J. Vegas Olmos. "QRoNS: Quantum Resilience over IPsec Tunnels for Network Slicing." In: *Electronics*, 14 (21). (Jan. 2025). pp. 4234. issn: 2079-9292. doi: 10.3390/electronics14214234. url: https://www.mdpi.com/2079-9292/14/21/4234.

[35] A. Ivanov, N. Dryden, T. Ben-Nun, S. Li, and T. Hoefler. *Data Movement Is All You Need: A Case Study on Optimizing Transformers*. (Nov. 2021). doi: 10.48550/arXiv.2007.00072. url: http://arxiv.org/abs/2007.00072.

[36] A. Ivanov, B. Rothenberger, A. Dethise, M. Canini, T. Hoefler, and A. Perrig. "SAGE: Software-based Attestation for GPU Execution." (2023). pp. 485–499. url: https://www.usenix.org/conference/atc23/presentation/ivanov.

[37] I. Jang, A. Tang, T. Kim, S. Sethumadhavan, and J. Huh. "Heterogeneous Isolated Execution for Commodity GPUs." In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. (Apr. 2019). pp. 455–468. doi: 10.1145/3297858.3304021. url: https://dl.acm.org/doi/10.1145/3297858.3304021.

[38] J. Jiang, J. Qi, T. Shen, X. Chen, S. Zhao, S. Wang, L. Chen, G. Zhang, X. Luo, and H. Cui. "CRONUS: Fault-isolated, Secure and High-performance Heterogeneous Computing for Trusted Execution Environments: 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)." In: *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. (Oct. 2022). pp. 124–143. issn: 978-1-6654-7428-3.

[39] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. *Mixtral of Experts*. (Jan. 2024). doi: 10.48550/arXiv.2401.04088. url: http://arxiv.org/abs/2401.04088.

[40] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen. *Internet Key Exchange Protocol Version 2 (IKEv2)*. (Oct. 2014). doi: 10.17487/RFC7296. url: https://datatracker.ietf.org/doc/rfc7296.

[41] S. Kent. *IP Encapsulating Security Payload (ESP)*. (Dec. 2005). doi: 10.17487/RFC4303. url: https://datatracker.ietf.org/doc/rfc4303.

[42] A. Kalia, M. Kaminsky, and D. G. Andersen. "Design Guidelines for High Performance RDMA Systems." (2016). pp. 437–450. url: https://www.usenix.org/conference/atc16/technical-sessions/presentation/kalia.

[43] T. Knauth, M. Steiner, S. Chakrabarti, L. Lei, C. Xing, and M. Vij. *Integrating Remote Attestation with Transport Layer Security*. (July 2019). doi: 10.48550/arXiv.1801.05863. url: http://arxiv.org/abs/1801.05863.

[44] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. "Reducing Activation Recomputation in Large Transformer Models." In: *Proceedings of Machine Learning and Systems*, 5. (Mar. 2023). pp. 341–353. url: https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html.

[45] O. Kwon, Y. Kim, J. Huh, and H. Yoon. "ZeroKernel: Secure Context-Isolated Execution on Commodity GPUs." In: *IEEE Transactions on Dependable and Secure Computing*, 18 (4). (July 2021). pp. 1974–1988. issn: 1941-0018. doi: 10.1109/TDSC.2019.2946250. url: https://ieeexplore.ieee.org/document/8863993.

[46] J. Lawley. *Understanding performance of PCI express systems*. (Oct. 2014). url: https://docs.amd.com/v/u/en-US/wp350.

[47] J. Lee, Y. Wang, R. Rajat, and M. Annavaram. *Characterization of GPU TEE Overheads in Distributed Data Parallel ML Training*. (Aug. 2025). doi: 10.48550/arXiv.2501.11771. url: http://arxiv.org/abs/2501.11771.

[48] S. Lee, J. Drean, Y. Tan, and M. Peinado. "IOValve: Leakage-Free I/O Sandbox for Large-Scale Untrusted Data Processing." In: *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. (Nov. 2025). pp. 4454–4468. doi: 10.1145/3719027.3765121. url: https://dl.acm.org/doi/10.1145/3719027.3765121.

[49] Y. Liao, J. Wu, W. Lu, H. Lu, X. Li, and G. Yan. "SNO: Securing Network Function Offloading on FPGA-based SmartNICs in Untrusted Clouds." In: *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. (Oct. 2025). pp. 1–9. issn: 1558-2434. doi: 10.1109/ICCAD66269.2025.11240873. url: https://ieeexplore.ieee.org/document/11240873.

[50] M. Li, S. Srivastava, and M. Yan. *Bridge the Future: High-Performance Networks in Confidential VMs without Trusted I/O devices*. (Mar. 2024). doi: 10.48550/arXiv.2403.03360. url: http://arxiv.org/abs/2403.03360.

[51] H. Mai, J. Zhao, H. Zheng, Y. Zhao, Z. Liu, M. Gao, C. Wang, H. Cui, X. Feng, and C. Kozyrakis. "Honeycomb: Secure and Efficient GPU Executions via Static Validation." (2023). pp. 155–172. url: https://www.usenix.org/conference/osdi23/presentation/mai.

[52] C. Man, J. Park, H. Wu, H. Xu, S. Sridharan, and T. Krishna. *Scalable Synthesis of distributed LLM workloads through Symbolic Tensor Graphs*. (June 2026). doi: 10.48550/arXiv.2511.10480. url: http://arxiv.org/abs/2511.10480.

[53] A. Mohan, M. Ye, H. Franke, M. Srivatsa, Z. Liu, and N. M. Gonzalez. "Securing AI Inference in the Cloud: Is CPU-GPU Confidential Computing Ready?." In: *2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*. (July 2024). pp. 164–175. issn: 2159-6190. doi: 10.1109/CLOUD62652.2024.00028. url: https://ieeexplore.ieee.org/abstract/document/10643934.

[54] S. Na, J. Kim, S. Lee, and J. Huh. "Supporting Secure Multi-GPU Computing with Dynamic and Batched Metadata Management." In: *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. (Mar. 2024). pp. 204–217. issn: 2378-203X. doi: 10.1109/HPCA57654.2024.00025. url: https://ieeexplore.ieee.org/document/10476487.

[55] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. "Efficient large-scale language model training on GPU clusters using megatron-LM." In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. (Nov. 2021). pp. 1–15. doi: 10.1145/3458817.3476209. url: https://dl.acm.org/doi/10.1145/3458817.3476209.

[56] R. Neugebauer, G. Antichi, J. F. Zazo, Y. Audzevich, S. López-Buedo, and A. W. Moore. "Understanding PCIe performance for end host networking." In: *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. (Aug. 2018). pp. 327–341. doi: 10.1145/3230543.3230560. url: https://dl.acm.org/doi/10.1145/3230543.3230560.

[57] S. Novakovic, Y. Shan, A. Kolli, M. Cui, Y. Zhang, H. Eran, L. Liss, M. Wei, D. Tsafrir, and M. Aguilera. *Storm: a fast transactional dataplane for remote data structures*. (Feb. 2019). doi: 10.48550/arXiv.1902.02411. url: http://arxiv.org/abs/1902.02411.

[58] NVIDIA. *NVIDIA secure AI with Blackwell and Hopper GPUs*. (Aug. 2025). url: https://docs.nvidia.com/nvidia-secure-ai-with-blackwell-and-hopper-gpus-whitepaper.pdf.

[59] E. Park, T. Thans, V. K. Kalidasan, Q. Hu, and W. Xiong. "An Empirical Study of LLM Serving in Confidential GPUs." In: *2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. (Apr. 2026). pp. 311–322. issn: 2766-0486. doi: 10.1109/ISPASS69572.2026.00038. url: https://ieeexplore.ieee.org/abstract/document/11527301.

[60] J. Park, Y. Wang, H. Xu, H. Wu, M. Li, and T. Krishna. "SCALE: Tackling Communication Bottlenecks in Confidential Distributed Machine Learning." In: *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*.

(Jan. 2026). pp. 1–14. issn: 2378-203X. doi: 10.1109/HPCA68181.2026.11408582. url: https://ieeexplore.ieee.org/document/11408582.

[61] S. Pati, S. Aga, M. Islam, N. Jayasena, and M. D. Sinclair. *Computation vs. Communication Scaling for Future Transformers on Future Hardware*. (May 2023). doi: 10.48550/arXiv.2302.02825. url: http://arxiv.org/abs/2302.02825.

[62] PCI-SIG. *Integrity and Data Encryption (IDE) Revision A*. (Jan. 2022). url: https://pcisig.com/PCI%20Express/ECN/Base/IntegrityandDataEncryption_A.

[63] PCI-SIG. *TEE Device Interface Security Protocol (TDISP)*. (Aug. 2022). url: https://pcisig.com/PCI%20Express/ECN/Base/TEEDeviceInterfaceSecurityProtocol.

[64] PCI-SIG. *PCI Express Base Specification*. (July 2023). url: https://pcisig.com/PCIExpress/Specs/Base/_6.1.

[65] R. Poddar, G. Ananthanarayanan, S. Setty, S. Volos, and R. A. Popa. "Visor: Privacy-Preserving Video Analytics as a Cloud Service." (2020). pp. 1039–1056. url: https://www.usenix.org/conference/usenixsecurity20/presentation/poddar.

[66] S. Potluri, K. Hamidouche, A. Venkatesh, D. Bureddy, and D. K. Panda. "Efficient Inter-node MPI Communication Using GPUDirect RDMA for InfiniBand Clusters with NVIDIA GPUs." In: *2013 42nd International Conference on Parallel Processing*. (Oct. 2013). pp. 80–89. issn: 2332-5690. doi: 10.1109/ICPP.2013.17. url: https://ieeexplore.ieee.org/document/6687341.

[67] P. Patarasuk and X. Yuan. "Bandwidth optimal all-reduce algorithms for clusters of workstations." In: *Journal of Parallel and Distributed Computing*, 69 (2). (Feb. 2009). pp. 117–124. issn: 0743-7315. doi: 10.1016/j.jpdc.2008.09.002. url: https://www.sciencedirect.com/science/article/pii/S0743731508001767.

[68] Z. Ren, M. Fan, Z. Wang, J. Zhang, C. Zeng, Z. Huang, C. Hong, and K. Chen. "Accelerating Secure Collaborative Machine Learning with Protocol-Aware RDMA." (2024). pp. 2245–2261. url: https://www.usenix.org/conference/usenixsecurity24/presentation/ren.

[69] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. (July 2026). doi: 10.17487/RFC9846. url: https://datatracker.ietf.org/doc/rfc9846.

[70] B. Rothenberger, K. Taranov, A. Perrig, and T. Hoefler. "ReDMArk: Bypassing RDMA Security Mechanisms." (2021). pp. 4277–4292. url: https://www.usenix.org/conference/usenixsecurity21/presentation/rothenberger.

[71] M. Schneider, A. Dhar, I. Puddu, K. Kostiainen, and S. Capkun. "Composite Enclaves: Towards Disaggregated Trusted Execution." In: *IACR Transactions on Cryptographic Hardware and Embedded Systems*. (Nov. 2021). pp. 630–656. issn: 2569-2925. doi: 10.46586/tches.v2022.i1.630-656. url: http://arxiv.org/abs/2010.10416.

[72] M. Schneider, R. J. Masti, S. Shinde, S. Capkun, and R. Perez. *SoK: Hardware-supported Trusted Execution Environments*. (May 2022). doi: 10.48550/arXiv.2205.12742. url: http://arxiv.org/abs/2205.12742.

[73] K. Seo and S. Kent. *Security Architecture for the Internet Protocol*. (Dec. 2005). doi: 10.17487/RFC4301. url: https://datatracker.ietf.org/doc/rfc4301.

[74] S. Sridhara, A. Bertschi, B. Schlüter, M. Kuhne, F. Aliberti, and S. Shinde. "ACAI: Protecting Accelerator Execution with Arm Confidential Computing Architecture." (2024). pp. 3423–3440. url: https://www.usenix.org/conference/usenixsecurity24/presentation/sridhara.

[75] Y. Tan, C. Tan, Z. Mi, and H. Chen. *PipeLLM: Fast and Confidential Large Language Model Services with Speculative Pipelined Encryption*. (Nov. 2024). doi: 10.48550/arXiv.2411.03357. url: http://arxiv.org/abs/2411.03357.

[76] K. Taranov, B. Rothenberger, A. Perrig, and T. Hoefler. "sRDMA – Efficient NIC-based Authentication and Encryption for Remote Direct Memory Access." (2020). pp. 691–704. url: https://www.usenix.org/conference/atc20/presentation/taranov.

[77] N. I. of S. Technology (NIST), , M. J. Dworkin, E. Barker, J. R. Nechvatal, J. Foti, L. E. Bassham, E. Roback, and J. F. D. Jr. "Advanced Encryption Standard (AES)." In: *NIST*. (Nov. 2001). url: https://www.nist.gov/publications/advanced-encryption-standard-aes.

[78] R. Thakur, R. Rabenseifner, and W. Gropp. "Optimization of Collective Communication Operations in MPICH." In: *The International Journal of High Performance Computing Applications*, 19 (1). (Feb. 2005). pp. 49–66. issn: 1094-3420. doi: 10.1177/1094342005051521. url: https://doi.org/10.1177/1094342005051521.

[79] K. Vaswani, S. Volos, C. Fournet, A. N. Diaz, K. Gordon, B. Vembu, S. Webster, D. Chisnall, S. Kulkarni, G. Cunningham, R. Osborne, and D. Wilkinson. *Confidential Machine Learning within Graphcore IPUs*. (May 2022). doi: 10.48550/arXiv.2205.09005. url: http://arxiv.org/abs/2205.09005.

[80] S. Volos, K. Vaswani, and R. Bruno. "Graviton: Trusted Execution Environments on GPUs." (2018). pp. 681–696. url: https://www.usenix.org/conference/osdi18/presentation/volos.

[81] Y. Wang, R. Rajat, J. Lee, T. Tang, and M. Annavaram. *Fastrack: Fast IO for Secure ML using GPU TEEs*. (Oct. 2024). doi: 10.48550/arXiv.2410.15240. url: http://arxiv.org/abs/2410.15240.

[82] C. Wang, D. Tang, C. Ci, J. Huang, Y. Xu, F. Zhang, J. Cao, J. Song, S. Yan, T. Wei, and Z. He. "ccAI: A Compatible and Confidential System for AI Computing." In: *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*. (Oct.

2025). pp. 340–353. doi: 10.1145/3725843.3756104. url: https://dl.acm.org/doi/10.1145/3725843.3756104.

[83] C. Weinhold, M. U. Sardar, I. Mihalcea, Y. Deshpande, H. Tschofenig, Y. Sheffer, T. Fossati, and M. Roitzsch. "Separate but Together: Integrating Remote Attestation into TLS." (2025). pp. 1319–1326. url: https://www.usenix.org/conference/atc25/presentation/weinhold.

[84] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna. "ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-model Training at Scale." In: *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. (Apr. 2023). pp. 283–294. doi: 10.1109/ISPASS57527.2023.00035. url: http://arxiv.org/abs/2303.14006.

[85] C. Wang, F. Zhang, and Y. Deng. *CAGE: Complementing Arm CCA with GPU Extensions*. url: https://www.ndss-symposium.org/ndss-paper/cage-complementing-arm-cca-with-gpu-extensions/.

[86] J. Xing, K.-F. Hsu, Y. Qiu, Z. Yang, H. Liu, and A. Chen. "Bedrock: Programmable Network Support for Secure RDMA Systems." (2022). pp. 2585–2600. url: https://www.usenix.org/conference/usenixsecurity22/presentation/xing.

[87] Y. Yan, I. Karlin, and R. Grant. *Revealing NVIDIA Closed-Source Driver Command Streams for CPU-GPU Runtime Behavior Insight*. (Apr. 2026). doi: 10.48550/arXiv.2604.26889. url: http://arxiv.org/abs/2604.26889.

[88] Y. Yang, M. Sonji, and A. Jog. "Dissecting Performance Overheads of Confidential Computing on GPU-based Systems." In: *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. (May 2025). pp. 1–16. issn: 2766-0486. doi: 10.1109/ISPASS64960.2025.00020. url: https://ieeexplore.ieee.org/abstract/document/11096367.

[89] A. W. B. Yudha, J. Meyer, S. Yuan, H. Zhou, and Y. Solihin. "LITE: a low-cost practical inter-operable GPU TEE." In: *Proceedings of the 36th ACM International Conference on Supercomputing*. (June 2022). pp. 1–13. doi: 10.1145/3524059.3532361. url: https://dl.acm.org/doi/10.1145/3524059.3532361.

[90] H. Yin and K. Wang. *The Serialized Bridge: Understanding and Recovering LLM Serving Performance under Blackwell GPU Confidential Computing*. (July 2026). doi: 10.48550/arXiv.2606.23969. url: http://arxiv.org/abs/2606.23969.

[91] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, A. Desmaison, C. Balioglu, P. Damania, B. Nguyen, G. Chauhan, Y. Hao, A. Mathews, and S. Li. *PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel*. (Sept. 2023). doi: 10.48550/arXiv.2304.11277. url: http://arxiv.org/abs/2304.11277.

[92] Y. Zhou, M. Wilkening, J. Mickens, and M. Yu. "SmartNIC Security Isolation in the Cloud with S-NIC." In: *Proceedings of the Nineteenth European Conference on Computer Systems*. (Apr. 2024). pp. 851–869. doi: 10.1145/3627703.3650071. url: https://dl.acm.org/doi/10.1145/3627703.3650071.

[93] J. Zhu, R. Hou, X. Wang, W. Wang, J. Cao, L. Zhao, F. Yuan, P. Li, Z. Wang, B. Zhao, L. Zhang, and D. Meng. *Enabling Privacy-Preserving, Compute- and Data-Intensive Computing using Heterogeneous Trusted Execution Environment*. (Apr. 2019). doi: 10.48550/arXiv.1904.04782. url: http://arxiv.org/abs/1904.04782.

[94] J. Zhu, H. Yin, P. Deng, A. Almeida, and S. Zhou. *Confidential Computing on NVIDIA Hopper GPUs: A Performance Benchmark Study*. (Nov. 2024). doi: 10.48550/arXiv.2409.03992. url: http://arxiv.org/abs/2409.03992.